Supplemental information

ROBERT: Bridging the Gap between Machine Learning and Chemistry

David Dalmau, Juan V. Alegre Requena

Table of Contents

Installation of the Program

This section provides information regarding the program installation. Additional instructions for users with limited experience in Python are available in the “Installation / Users with no Python experience” section on our Read the Docs page.[1] Before installing ROBERT, ensure Anaconda is installed. Subsequently, execute the following command lines in an Anaconda prompt (Windows) or terminal (macOS and Linux):

```
conda create -n robert python=3.10
conda activate robert
```

The creation of the environment with the “conda create” command is required only once. We used Python 3.10 as the model version, but other versions (i.e., 3.7, 3.9, etc.) work similarly. In Windows, Anaconda Prompts were used as the terminals (available after installing Anaconda for Windows), while default terminals were used in macOS and Linux systems. Subsequently, the following command line was used to install ROBERT:

```
conda install -c conda-forge robert
```

When possible, we recommend installing the scikit-learn-intelex code accelerator:

```
pip install scikit-learn-intelex    (only for compatible systems)
```

Table S1 shows the researchers who participated in the installation test, along with the specifications of the machines used and the time required. In all cases, the installation of the program using “conda install” took less than two minutes. After ROBERT has been installed, the program is ready to use. Multiple examples on how to use the program are included on our Read the Docs webpage.[1] *It is important to ensure that the robert environment is activated in the terminal before executing the program.*

Table S1. Installation times required for different program testers using the “conda install -c conda-forge robert" command.

Tester	Group (institution)	Installation time (s)	OS (number of processors)
David Dalmau (author)	Alegre (CSIC-ISQCH)	A. 42 s	A. Windows 10 10.0.19045 (8)
Juan V. Alegre-Requena (author)	Alegre (CSIC-ISQCH)	A. 89 s	Windows 11 10.0.22621 (8)
David Valiente	Optoelectronic devices (Miguel Hernández University)	60 s	macOS Ventura 13.4.1 (8)
Íñigo Iribarren	Trujillo (Trinity College Dublin)	110 s	Arch Linux 6.5.3-arch1-1 (8)
Heidi Klem	Paton (Colorado State University)	84 s	MacOS Ventura Version 13.5.2 (8)
Guilian Luchini	Bristol Myers Squibb	43 s	macOS Ventura 13.4.1 (8)
Alex Platt	Paton (Colorado State University)	35 s	macOS Ventura 13.4.1 (8)
Xinchun Ran	Yang (Vanderbilt University)	A. 30 s B. 60 s	A. macOS Ventura 13.5 (8) B. Centos HPC (8)
Oliver Lee	Zysman-Colman (University of St. Andrews)	75 s	Linux Mint 21.1 Cinnamon (12)

Impact of scikit-learn-intelex

Intel® Extension for Scikit-learn (scikit-learn-intelex)[2] is a code accelerator that provides speed-ups of over 10-100X across a variety of scikit-learn[3] functions. Nevertheless, some computers may not be compatible with this accelerator, and on such computers, the original scikit-learn code runs by default. We carried out a benchmarking study using examples A-F from Figure 3 to identify the differences between ROBERT jobs conducted with and without scikit-learn-intelex (Table S2). This study revealed similar root mean squared error (RMSE) and score results when using scikit-learn-intelex compared to the standard scikit-learn version. However, scikit-learn-intelex showed faster times in five of the examples (A, B, D, E and F). For this reason, we recommend users to install this package since very similar results can be achieved while

the jobs typically require less execution time. All the jobs performed with ROBERT in this manuscript used scikit-learn-intelex.

Table S2. Times, RMSE, and ROBERT scores obtained in examples A-F of Figure 3 with and without the scikit-learn-intelex code accelerator.

Example	scikit-learn-intelex	Time (s)	Error (RMSE)	ROBERT Score
A	Yes	53.40	0,91	8
	No	96.53	0.93	9
B	Yes	53.10	0.12	8
	No	73.73	0.12	8
C	Yes	1221.10*	6.5	9
	No	1172.64	4.5	8
D	Yes	2764.29	2.6e+05	7
	No	3231.99	2.6e+05	7
E	Yes	4237.98	1.6	10
	No	4339.60	1.6	10
F	Yes	16657.77	0.13**	10
	No	17029.78	0.13**	10

* The use of scikit-learn-intelex did not decrease the execution time.

** Accuracy (classification) = 0.87.

Impact of the Correlation Filter

We performed a benchmarking study using examples A-F from Figure 3 to assess the impact of the correlation filter in the CURATE module. We examined the errors of the best models with and without the correlation filter, along with the number of descriptors employed by the algorithms (Table S3). While there are instances where deactivating the correlation filter reduces the error (A-C), there are also cases where the error either remains constant or increases, even with the inclusion of additional descriptors (D-F).

Therefore, we have chosen to keep the correlation filter as the default option, as it normally reduces the descriptors used and enhances the human interpretability of the models.

Table S3. Error (MAE for A, D and E, RMSE for B and C, and accuracy for F) and ROBERT scores obtained in examples A-F of Figure 3 with and without the correlation filter of the CURATE module. The correlation filter can be disabled using "--corr_filter False" in the command line.

Example	Correlation filter	Error	ROBERT Score	Descriptors used
A	Yes	0.72	8	1
	No	0.68	8	2
B	Yes	0.12	8	3
	No	0.077	9	2
C	Yes	6.5	9	2
	No	6.2	9	2
D	Yes	1.9e+05	7	11
	No	1.9e+05	7	12
E	Yes	1.2	10	32
	No	1.3	9	101
F	Yes	0.13*	10	60
	No	0.14**	10	84

* Accuracy (classification) = 0.87.

** Accuracy (classification) = 0.86

Benchmarking of Number of Random Seeds and Epochs in GENERATE

To determine the optimal trade-off between precision and computational efficiency, we conducted a benchmarking study. This study systematically varied the quantity of seeds used in the algorithms and the number of epochs employed in hyperoptimization function during the model screening performed in the

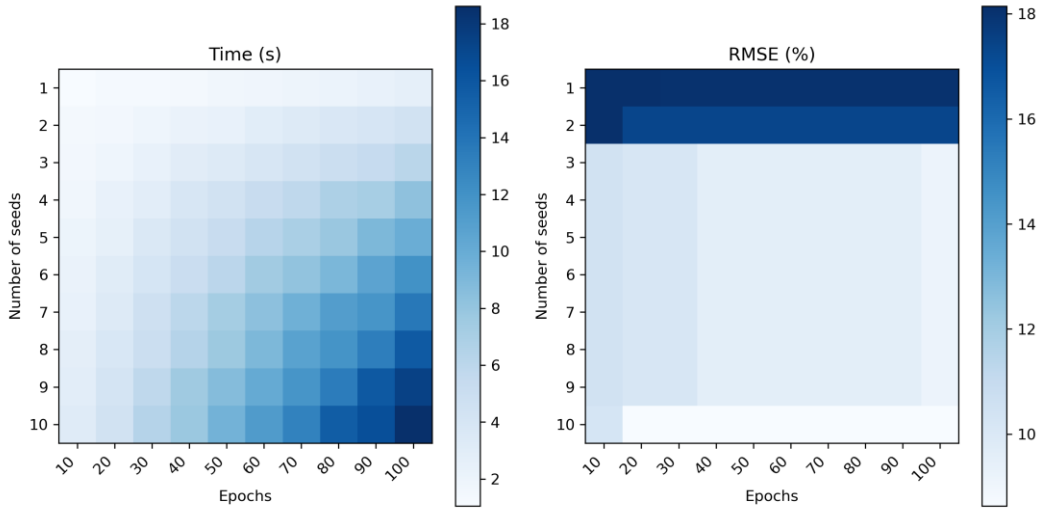
GENERATE module. This thorough exploration included examples A-E from Figure 3 and involved different algorithms and training sizes.

Within the program, we introduced three precision levels: "low", "mid" and "high", and these levels are selected with the ("--generate_acc") keyword. These options depend on specific combinations of seeds and epochs, which can be categorized as follows:

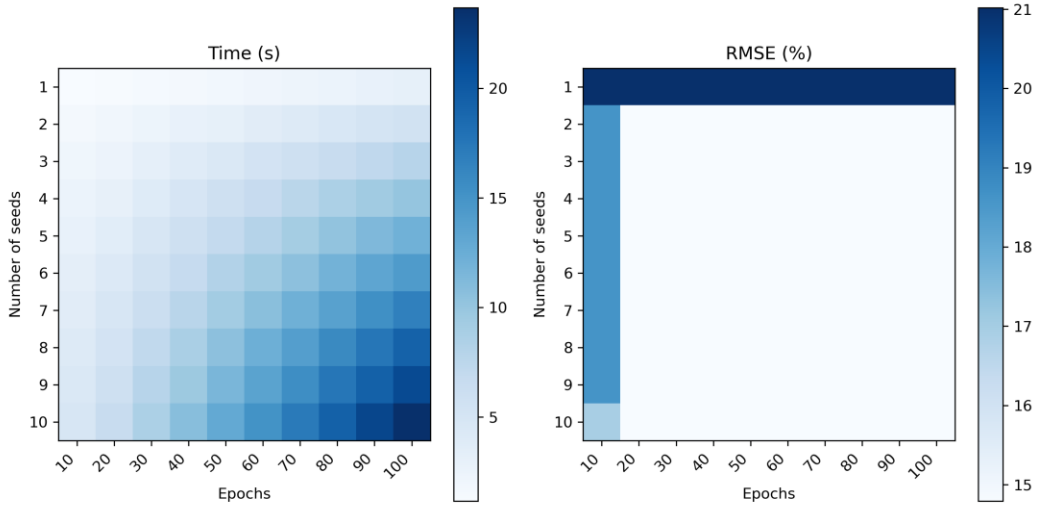
- Low: This setting represents the fastest but least accurate protocol (seed = [0, 8, 19], epochs = 20).
- Mid (default): This option strikes a balance between speed and accuracy, offering intermediate precision (seed = [0, 8, 19, 43, 70, 233], epochs = 40).
- High: This setting is the slowest but most accurate protocol (seed = [0, 8, 19, 43, 70, 233, 1989, 9999, 20394, 3948301], epochs = 100).

We analyzed the results obtained with different accuracies, models and partition sizes. The tests included combinations of a RF model and 60% training size for examples A-E (Figure S1), GB, MVL and NN models with 60% training size for example A, and an RF model with 70%, 80%, and 90% training sizes for example B (Figure S2). We observed that a good balance between speed and accuracy was achieved with the “mid” option and, therefore, we used that option as the default accuracy level in ROBERT.

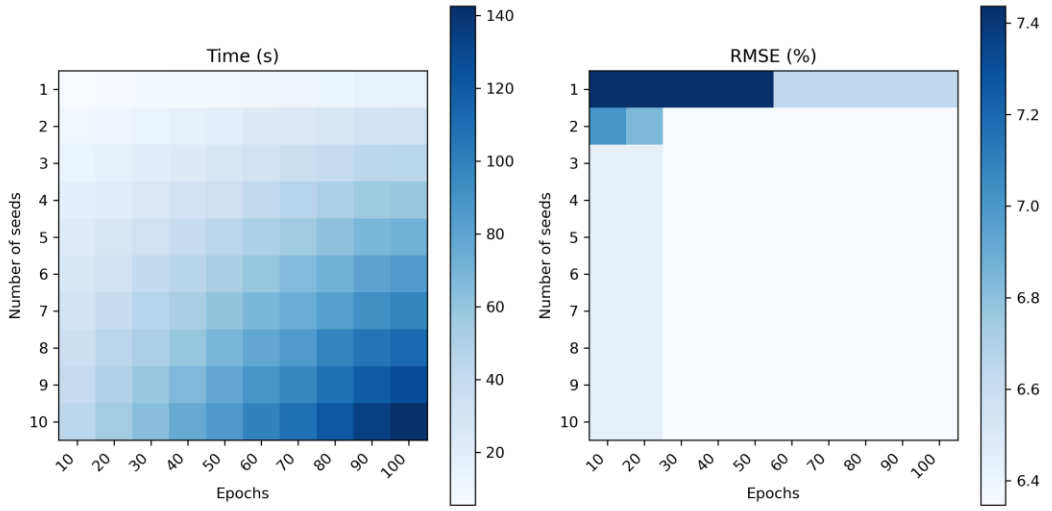
A.


Time (s)
RMSE (%)
Number of seeds
Epochs
1
2
3
4
5
6
7
8
9
10
10
20
30
40
50
60
70
80
90
100
2
4
6
8
10
12
14
16
18

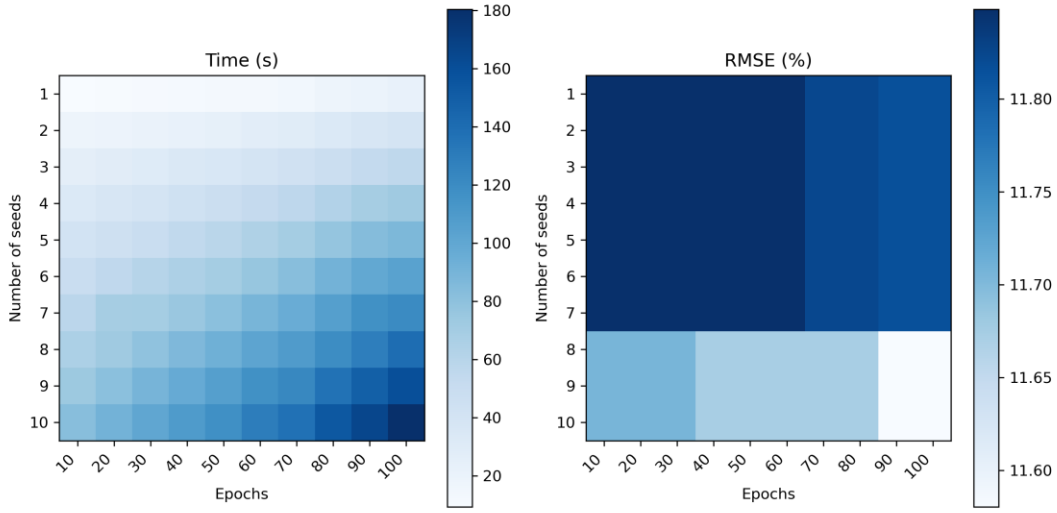
B.


Time (s)
RMSE (%)
Number of seeds
Epochs
1
2
3
4
5
6
7
8
9
10
10
20
30
40
50
60
70
80
90
100
5
10
15
20
15
16
17
18
19
20
21

C.


Time (s)
Number of seeds
Epochs
RMSE (%)
Number of seeds
Epochs

D.


Time (s)
Number of seeds
Epochs
RMSE (%)
Number of seeds
Epochs

E.

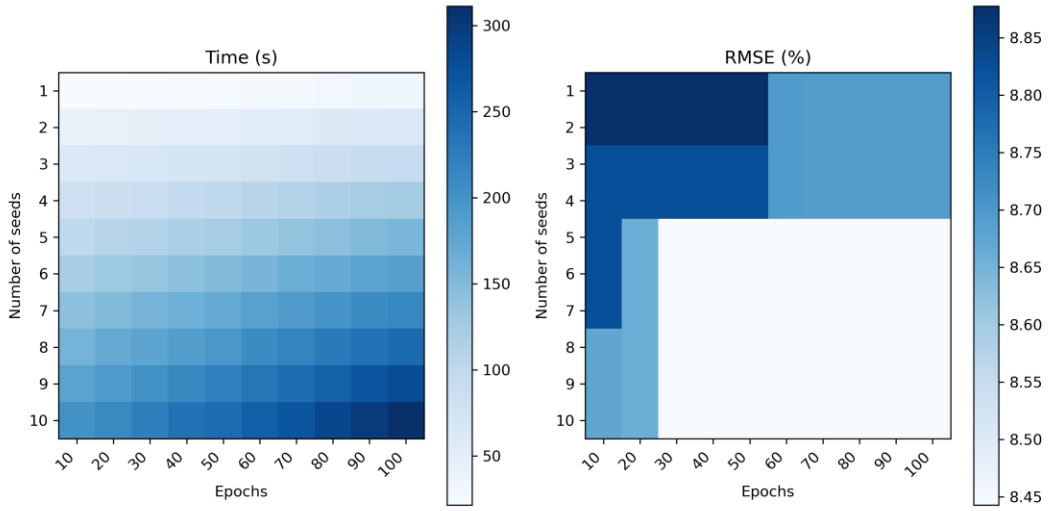
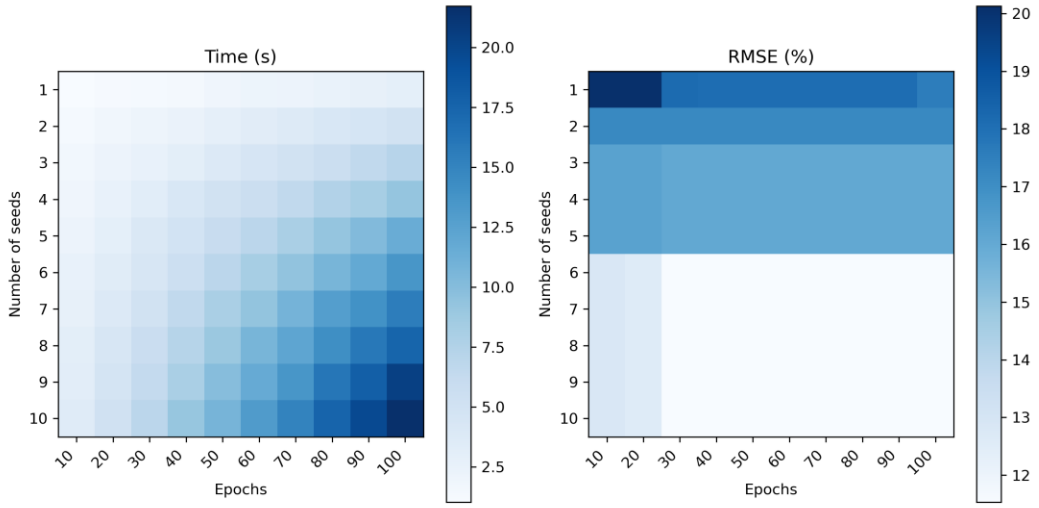
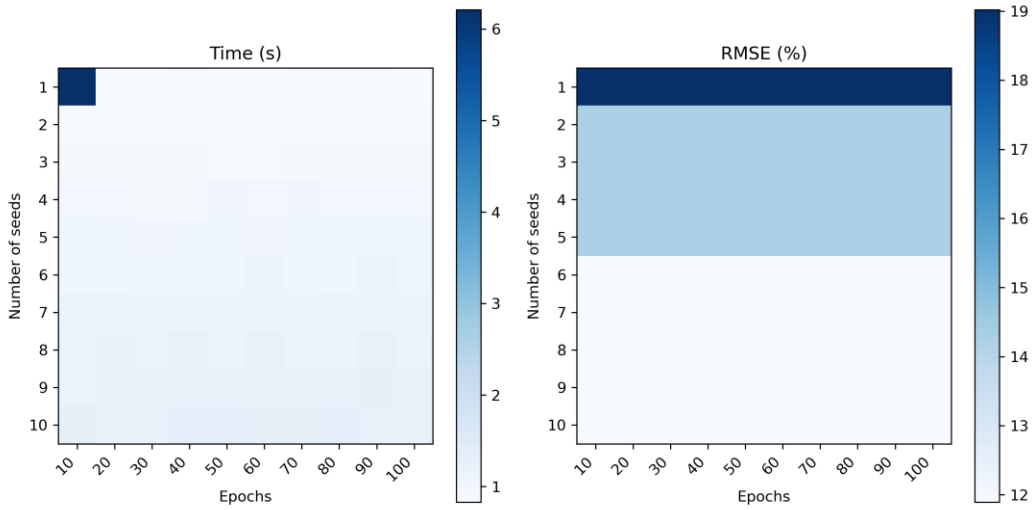


Figure S1. Execution times and RMSE obtained with a RF model and training size 60% of examples A-E.

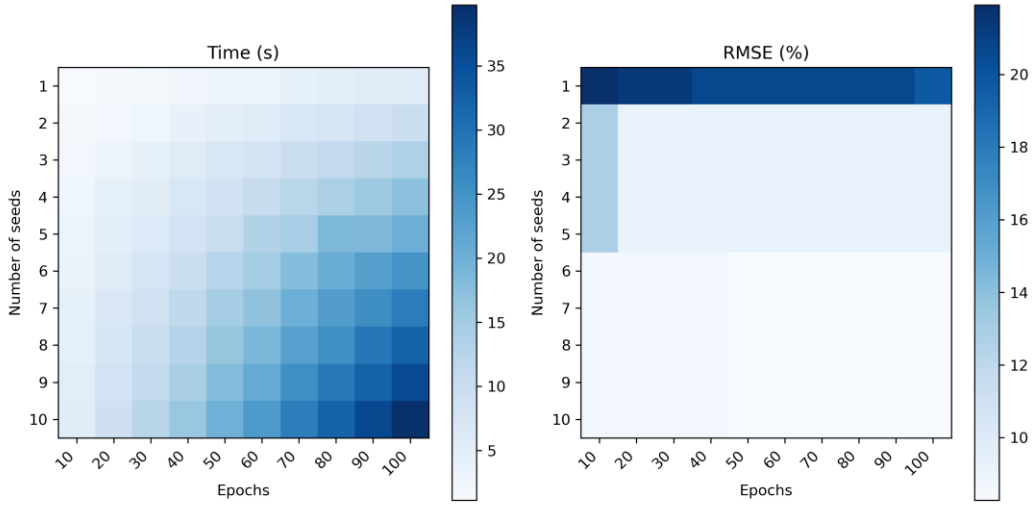
GB (60%, example A)



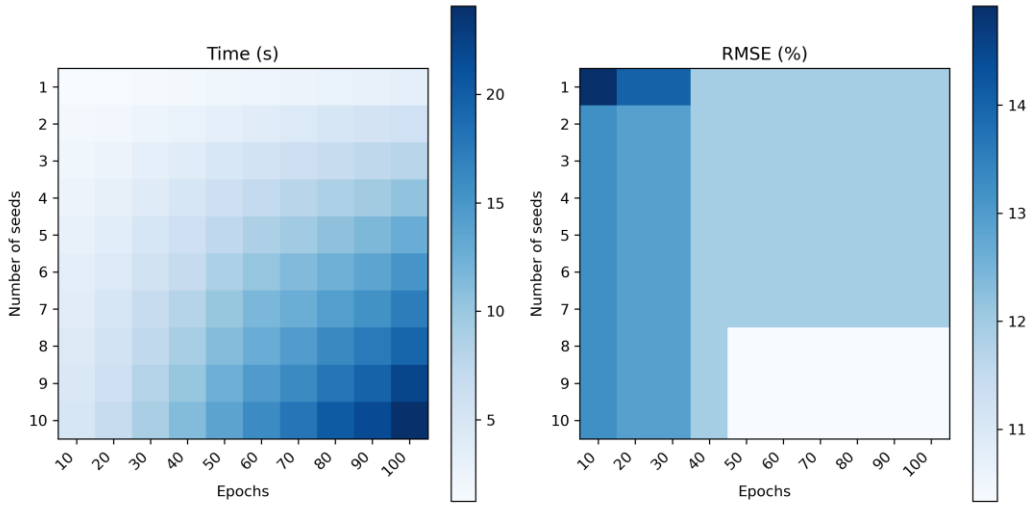
MVL (60%, example A)



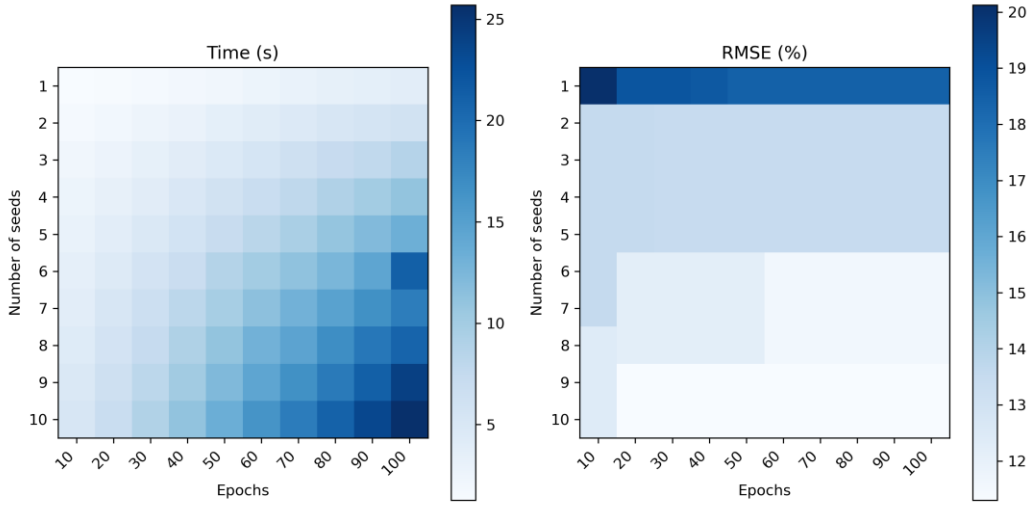
NN (60%, example A)



RF (70%, example B)



RF (80%, example B)



RF (90%, example B)

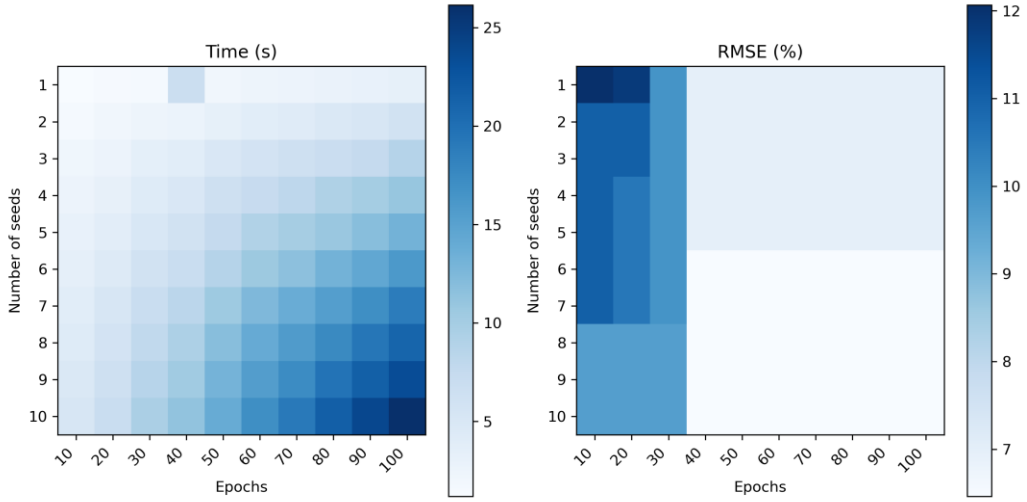


Figure S2. Benchmark with models GB (60% training size), MVL (60%) and NN (60%) for example A, and RF (70, 80 and 90%) for example B.

ROBERT Score

While experienced ML users can normally assess whether an ML workflow has yielded favorable results for valid reasons, inexperienced users may encounter difficulties in gauging the reliability of its predictive proficiency. It is widely recognized that ML algorithms can exhibit good metrics (i.e., coefficient of determination, R^2, mean absolute error, MAE, RMSE, and similar) in the validation set while harboring questionable predictive ability. For example, a user might assume that a model with good metrics is proficient at prediction, but that very model could yield comparably low errors when the y values are shuffled[4] or when using random numbers as descriptors,[5] casting doubt on its actual predictive validity.

For these reasons, we designed the ROBERT score, which is a rating out of 10 designed to provide users with insight into the predictive capabilities of the models selected by ROBERT. We attempted to rank the models using guidelines aligned with modern ML research (see below). For a more detailed explanation of best practices for designing robust ML models, see references[6,7].

Predictive ability towards the validation or external test set (2 points):

The R^2 (for regression) or accuracy (for classification) of the validation or external test set is employed to assess the predictive capabilities of the predictors found with ROBERT. These models undergo hyperoptimization using both a training set and a validation set. In cases where a test set is absent, metrics from the validation set are used instead. ROBERT, by default, allocates a sufficient number of data points in test/validation sets to ensure the generation of meaningful R^2/accuracy scores. Furthermore, comparable outcomes were achieved when alternative metrics like RMSE or MAE were employed.

Points	Condition
•• 2	$R^2 > 0.85$ (high correlation between predicted and measured y values)
•1	$0.85 > R^2 > 0.70$ (moderate correlation)
0	$R^2 < 0.70$ (low correlation)

Proportion of datapoints vs descriptors (2 points):

The ratio of datapoints to descriptors used during model hyperoptimization (in the train and validation sets) stands as another crucial parameter. Lower ratios result in simpler predictors that are more human-interpretable. The extensive literature on ML modeling offers numerous suggested ratios, and we endeavored to select reasonable parameters in accordance with previous recommendations.

Points	Condition
•• 2	Datapoints:descriptors ratio > 10:1 (reasonably low amount of descriptors)
• 1	10:1 > ratio > 3:1 (moderate amount)
0	Ratio < 3:1 (too many descriptors)

Passing VERIFY tests (4 points):

The tests conducted within the VERIFY module are also regarded as score indicators. The 5-fold cross-validation test guarantees the meaningfulness of the chosen data partition and guards against data overfitting. The y-mean and y-shuffle tests are valuable in identifying overfitted and underfitted models. Finally, the one-hot test identifies predictors that are insensitive to specific values but instead focus on the presence of such values (i.e., reaction datasets filled with 0s where compounds are not used).

Points	Condition
• 1	Each of the VERIFY tests passed (up to •••• 4 points)

Number of outliers in the validation set (only for regression, 2 points):

The count of outliers in the validation set is determined by analyzing the prediction errors against the mean and standard deviation (SD) of errors in the training set. The formula employed to compute the errors of the validation set in terms of SD units compared to the training set errors is as follows:

SD (valid. point) = [Error (valid. point) - mean error (training set)] / SD (training set)

By default, ROBERT adopts a t-value of 2 to identify outliers, which according to Gaussian distribution principles should lead to approximately 5% of outliers. If the validation set exhibits a high number of outliers, it could indicate overfitting in the training set or an unbalanced distribution of points within the validation set.

Points	Condition
•• 2	Outliers < 7.5% (close to a normal distribution of errors in the valid. set)
• 1	7.5% < outliers < 15% (not that far from a normal distribution of errors)

Points	Condition
0	Outliers > 15% (far from a normal distribution of errors)

Extra points for VERIFY tests (only for classification, 2 points):

As outliers are not calculated for classification problems, additional points are awarded for passing the y-mean and y-shuffle VERIFY tests. These specific tests were selected due to their significance in identifying potential shortcomings in the predictive capacity of the models.

Points	Condition
• 1	Each y-mean and y-shuffle tests passed (up to •• 2 points)

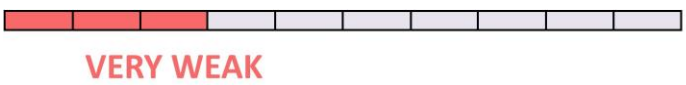
Score ranges

Some of the most common reasons for getting low ROBERT scores are:

- Unbalanced datasets (i.e., too many points in a region, too few in others)
- Including too few datapoints
- Including too few descriptors
- Overfitted and underfitted models

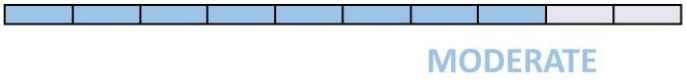
Different causes that might be affecting the score are included in the ROBERT score section of the PDF report. The score ranges are divided into four categories:

Very weak (very unreliable models)

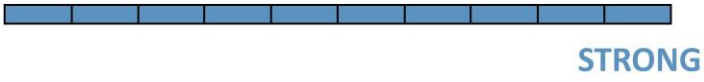


Weak (unreliable models)

Moderate (somewhat reliable models)



Strong (reliable models)



Benchmarking of SMILES Workflows

The options used in the SMILES workflows were optimized using examples A and B from Figure 4, including the number of conformers generated in AQME-CSEARCH, the optimization levels of xTB in AQME-QDESCP and the accuracy of the final xTB single-point calculations in AQME-QDESCP (Table S4).

Table S4. Error (RMSE) and ROBERT Score comparison varying the number of conformers, xTB single-point calculation accuracy, and xTB optimization level in examples A and B of Figure 4.

Example	n conformers	Acc. xTB	Opt. CREST	Error test set (RMSE)	ROBERT Score
A	5	5	Extreme	0.77	8
	10	1	Normal	0.66	8
	5	5	Normal	0.72	10
B	10	1	Normal	1.6	10
	5	5	Normal	1.5	10

When repeating end-to-end workflows starting from SMILES strings, it may not be possible to exactly reproduce the results due to subtle differences in the generated xTB descriptors (0.1% changes in the vast majority of cases, Figure S3). However, the resulting ROBERT scores and model accuracies are very similar between different ROBERT jobs (Table S5). In such cases, the PDF report recommends that authors upload the descriptor database created (i.e., AQME-ROBERT_FILENAME.csv) to facilitate the reproduction of results by other researchers. Users should be careful when using SMILES workflows with less than 50 datapoints since the results might change significantly between different runs of ROBERT. Users should execute a few jobs in parallel to ensure this is not their case.

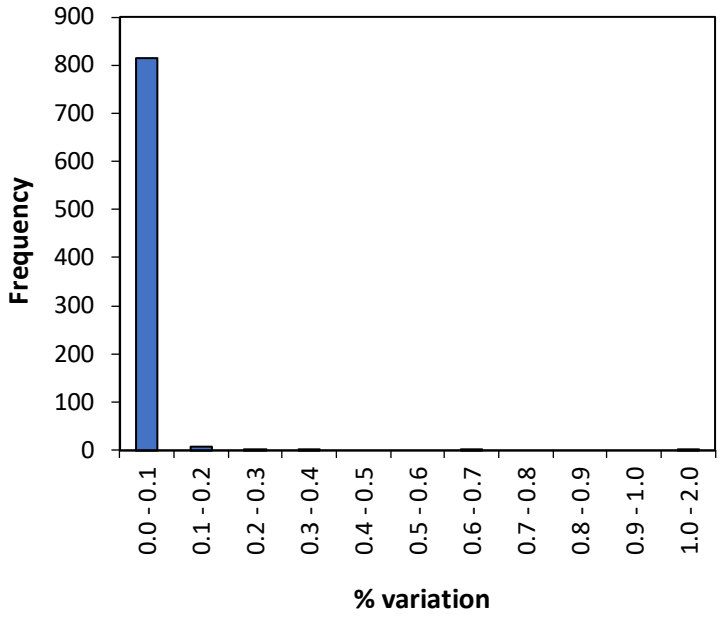


Figure S3. Distribution of maximum variations of descriptor values in absolute percentage among three different runs for the SMILES-based Pd discovery workflow. The maximum variation is determined as the highest difference between values obtained in the three runs (in %), encompassing over 800 combinations of data points and descriptors.

Table S5. RMSE and ROBERT Score comparison between runs for examples A-B of Figure 4.

Example	Run	Error test set (RMSE)	ROBERT Score
A	1	0.72	10
	2	0.72	10
	3	0.72	10
B	1	1.5	10
	2	1.6	9

ROBERT Modules

ROBERT comprises several modules that can be used either as part of a workflow or independently. These modules encompass: i) data curation (CURATE), responsible for removing irrelevant data, addressing missing values, and reducing the number of features; ii) model generation (GENERATE), which employs curated data for training and hyperoptimizing ML algorithms; iii) model verification (VERIFY), dedicated to analyzing the performance and reliability of the ML predictor; iv) prediction (PREDICT), applying the trained model to predict new data; and v) generating descriptors from SMILES (AQME).[8]

Data curation: the CURATE module

The CURATE module offers features to improve data quality. These include the removal of correlated variables, noise, and duplicated entries (Figure S4). These options are activated by default to reduce the complexity of the resulting predictors (Table S3), and users can set specific thresholds for the correlation and noise filters using the "thres_x" and "thres_y" parameters. This module also standardizes the values for each descriptor and replaces missing data points with zeros, removing any descriptors that have over 30% missing data. Additionally, the module facilitates the conversion of categorical data, transforming columns containing categorical variables into numerical or one-hot encoding values. For example, consider a variable that represents four types of carbon atoms (e.g., primary, secondary, tertiary, quaternary). This variable can be converted using the "categorical" command, offering the following options:

- “Numbers”: It assigns numerical values (e.g., 1, 2, 3, 4) to describe the different C atom types.

- "Onehot": It creates a separate descriptor for each C atom type using 0s and 1s to indicate their presence.

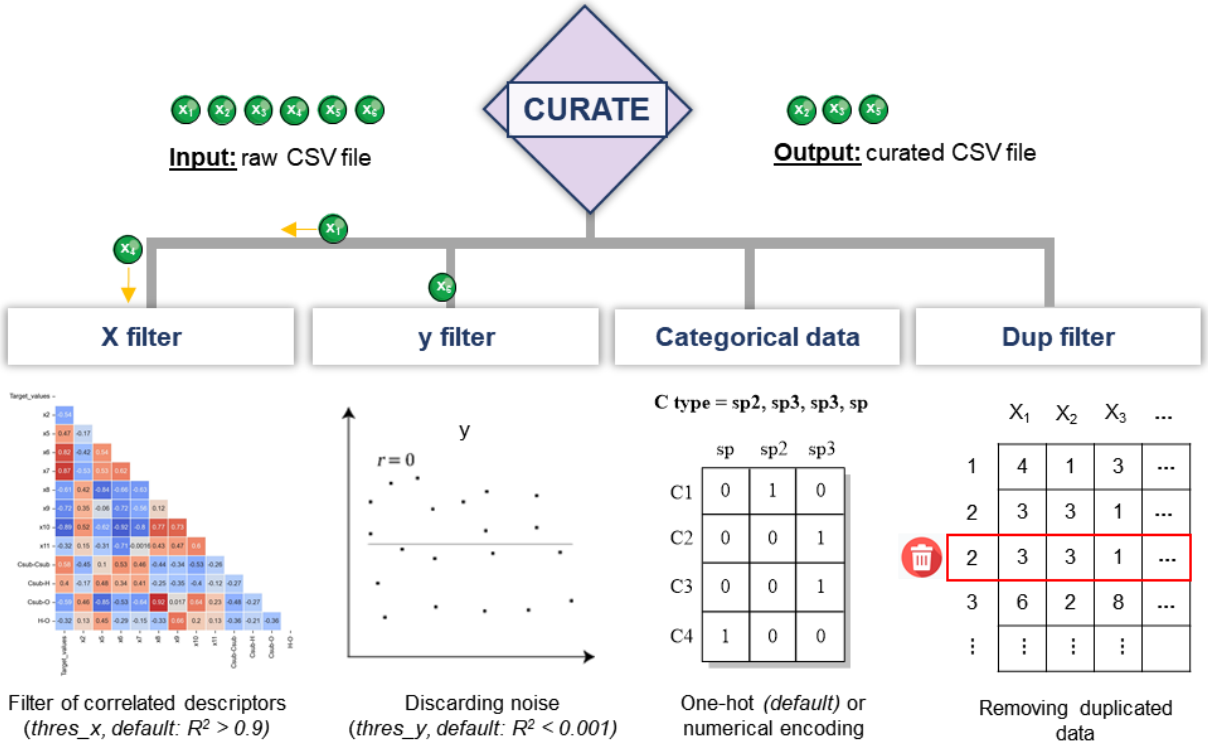


Figure S4. Overview, relevant options and defaults of the CURATE module.

Finding the optimal ML models: the GENERATE module

The GENERATE module performs an exploration of various combinations ML algorithms and partition sizes. It uses built-in ML models from scikit-learn[3] or its code accelerator, scikit-learn-intelex.[2] These models are hyperoptimized using the hyperopt Python module to find their optimal parameters. The determination of the number of epochs for hyperoptimization and random seeds of the models was achieved through exhaustive benchmarking (Figures S1-S2).

The software automatically generates a heatmap displaying RMSE values obtained from hyperoptimized algorithms (Figure S5). Furthermore, it performs permutation feature importance (PFI) analysis to identify the most influential descriptors and generate new models with only those descriptors. Users have the flexibility to fine-tune the PFI filter threshold using the "PFI_threshold" parameter. By default, this threshold

removes features that contribute less than 4% to the model's R^2. While this filter is activated by default, users can deactivate it by setting the "pfi_filter" option to False.

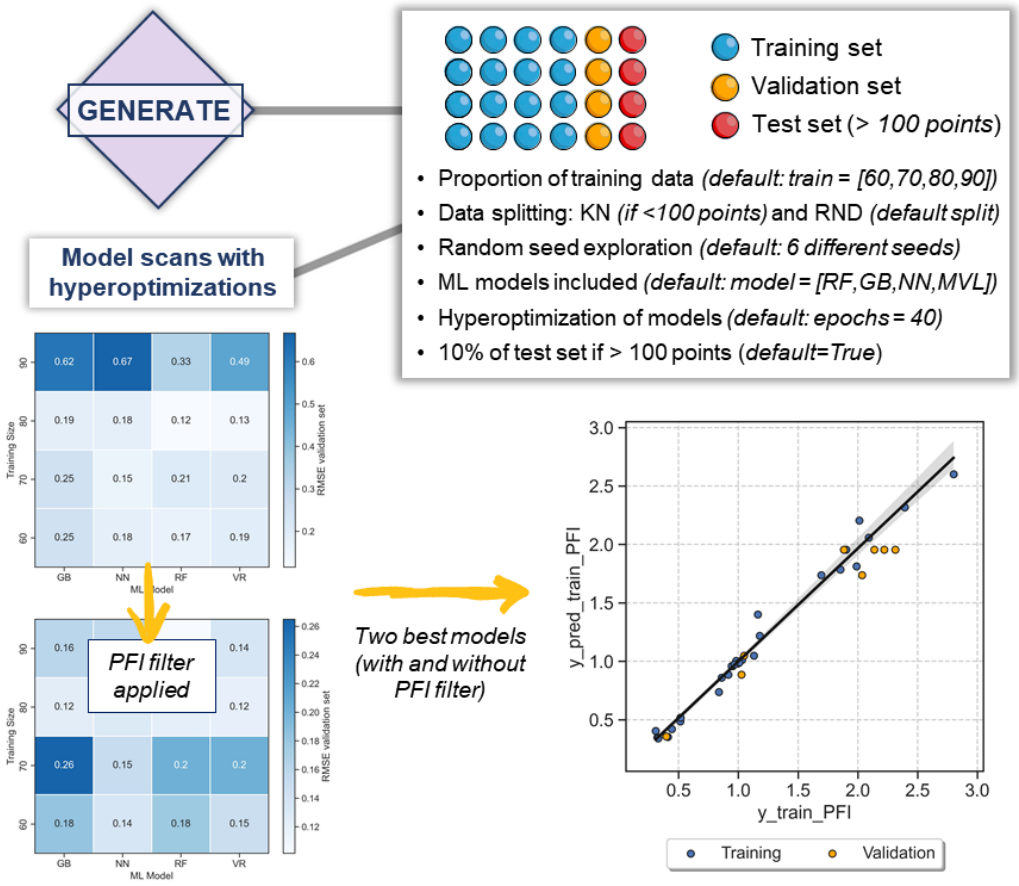


Figure S5. Overview, relevant options and defaults of the GENERATE module.

The model screening involves testing various partition sizes, which can be changed using the "train" keyword. Moreover, users can choose between two different modes for data splitting using the "split" option: "KN" (k-neighbors clustering-based) and "RND" (random). The selection of ML algorithms during screening is tuned through the "model" parameter, offering a range of popular options such as Random Forests (RF), Multivariate Linear Models (MVL),[9] Gradient Boosting (GB), Gaussian Process (GP), AdaBoost Regressor (AdaB),[10] MLP Regressor Neural Network (NN),[11] and Voting Regressor (VR).[12]

This module is designed to handle both regression and classification problems, optimizing either the RMSE (regression) or accuracy (classification) of the validation set during the hyperoptimization process. Users can adjust the error type for optimization with the "error_type" keyword, which offers the following options: RMSE, MAE, and R^2 for regression tasks, and Matthew's correlation coefficient (MCC), F1 score, and accuracy for classification tasks.

Evaluating the resulting models: the VERIFY module

The VERIFY module analyzes the performance and reliability of the ML predictors obtained in the GENERATE module (Figure S6). The following four tests are executed:

- y-mean test: Calculates the accuracy of the model when all the predicted y values are fixed to the mean of the measured y values (straight line when plotting measured vs predicted y values).
- y-shuffle test: Calculates the accuracy of the model after shuffling randomly all the measured y values.
- 5-fold cross-validation test: Calculates the accuracy of the model with a 5-fold cross-validation.
- One-hot test: Calculates the accuracy of the model when replacing all descriptors for 0s and 1s. If the x value is 0, the value will be 0, otherwise it will be 1.

The test-pass threshold is established through the 'thres_test' option, determined by the percentage of error difference (RMSE by default) relative to the original model. The 'kfold' parameter governs the number of folds utilized in cross-validation.

Lastly, the VERIFY module generates a donut plot summarizing the test outcomes. This visual representation distinguishes between passed and failed tests, using blue and red colors, respectively.

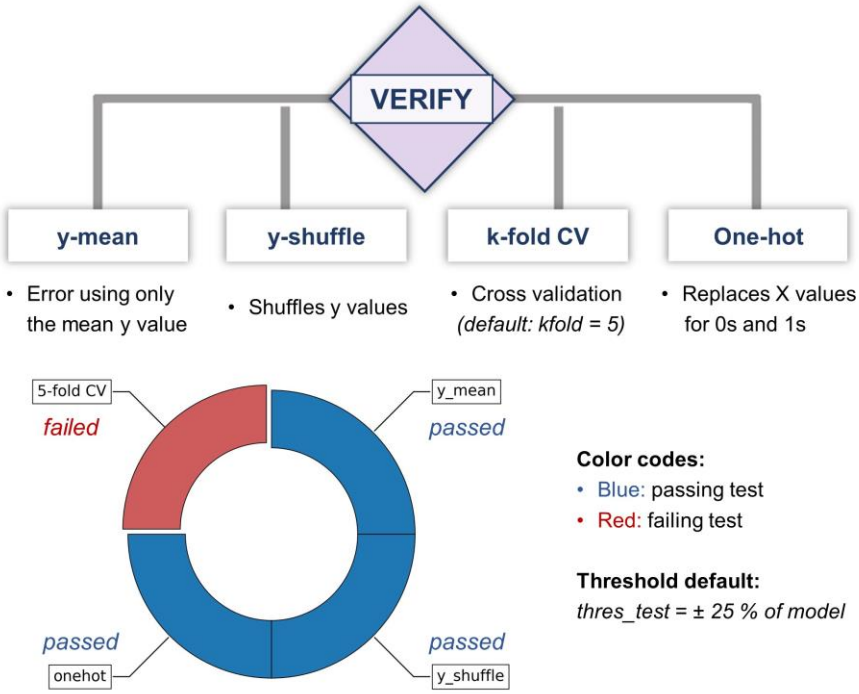


Figure S6. Overview, relevant options and defaults of the VERIFY module.

Generation of ML predictors: the PREDICT module

The PREDICT module uses models obtained in the GENERATE module to compute various metrics, including R2, MAE, and RMSE (regression), and accuracy, F1 score, and MCC (classification) (Figure S7). This module also enables predictions for an external test dataset, incorporating predictor metrics when measured y-values are available. In cases where measured y-values are absent, the module shows predicted y-values in the resulting PDF report and within the csv_test folder created inside the PREDICT main folder.

Furthermore, this module conducts feature importance analysis through PFI and SHAP methods, which analyze how descriptors impact model performance. The PREDICT module also identifies outliers by measuring the absolute errors between predicted and measured y values. The detection of outliers is based on the “t_value” option, defaulted to two and measured in SD units. This default t-value identifies outliers in predictions exhibiting errors surpassing two SDs (approx. 5% of a normal population).

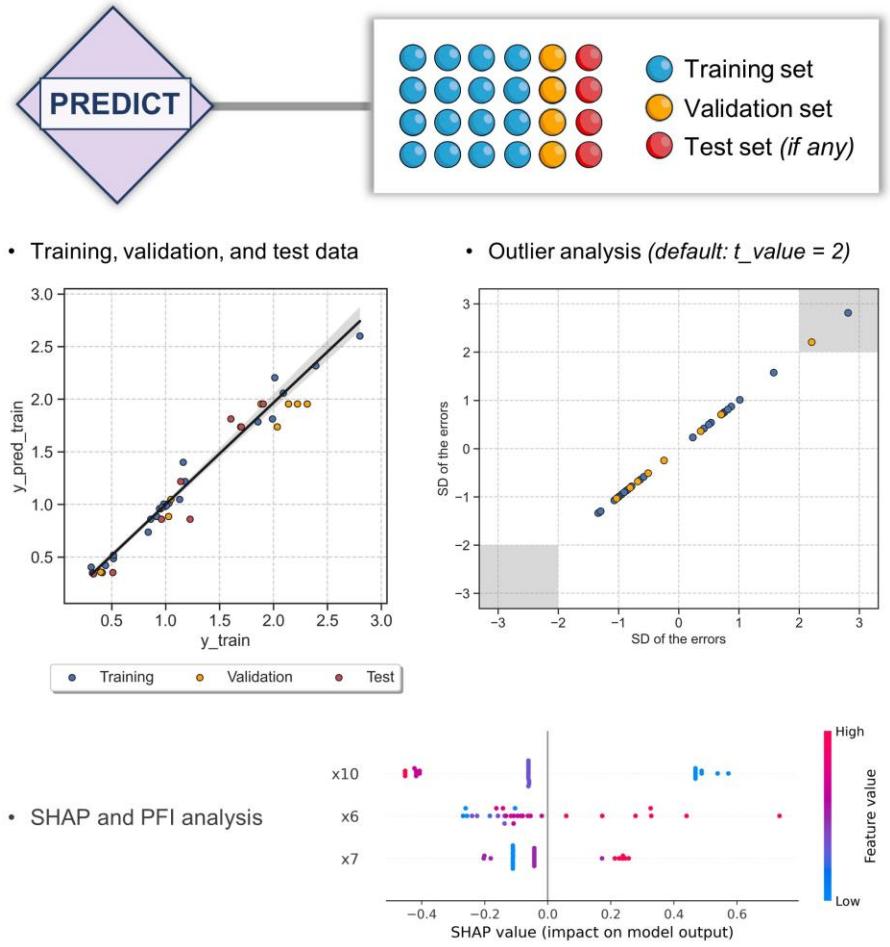


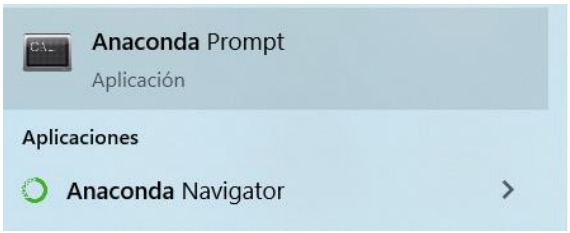
Figure S7. Overview, relevant options and defaults of the PREDICT module.

How to Reproduce the Results Presented in the Manuscript

The following steps detail how to reproduce the results for benchmark example A of Figure 3:

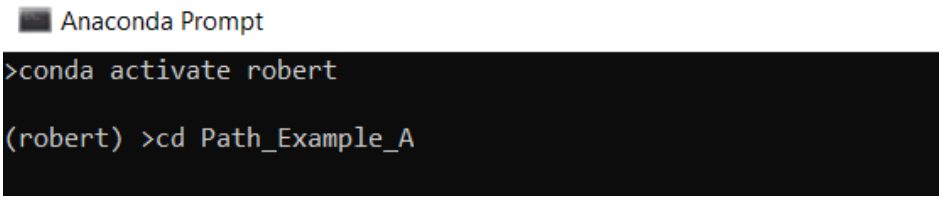
Requirements: Prior to replicating the results, users must ensure they have installed ROBERT and Anaconda, and have set up a new conda environment. Detailed steps for these procedures are available in the "Installation" section of the Read the Docs documentation[1] and in the "Installation of the Program" section of this ESI document. Typically, installing Anaconda and creating a conda environment takes approximately 5 minutes for non-expert users, while the installation of ROBERT requires only one or two minutes (Table S1).

Step 1: Open an Anaconda prompt (Windows) or a terminal with Anaconda installed (macOS and Linux).



Step 2: Activate the ROBERT environment with the command “conda activate robert".

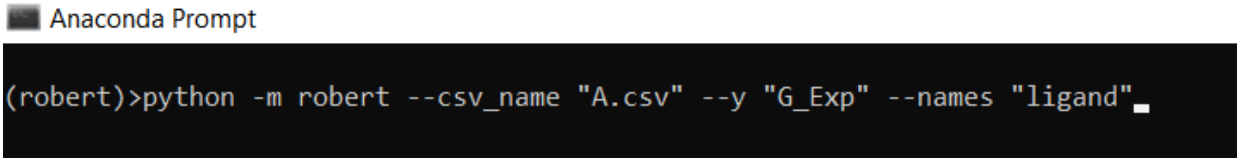
Step 3: Use the cd command to navigate to the example folder. This folder should contain the corresponding “A.csv” database and "ROBERT_report.pdf" file, and can be downloaded from the supporting information of the manuscript or our Zenodo repository.[13]



Step 4: Open the "ROBERT_report.pdf" file and copy the command line included in the reproducibility section, point 3. Then, paste this line into your conda terminal and execute it.

3. Run ROBERT using this command line in the folder with the CSV database:

python -m robert --csv_name "A.csv" --y "G_Exp" --names "ligand"



Results: After executing this command line, the ROBERT job should generate the results of example A in the working directory. More information is available in the “Examples” section of the online documentation.

Benchmarking with Six Examples

To perform the benchmark of the studies shown in Figure 3, we initially downloaded the databases from the corresponding supporting information files or GitHub repositories except for example E, in which the authors directly provided the database.

In four cases (A, B, C and F), the original publications used only two sets (training and validation/test), and we run ROBERT accordingly with two types of sets (training and validation). To adjust this setting, we incorporated the "--auto_test False" option in examples with over 100 data points to prevent the default creation of a third set (test) in ROBERT jobs. In all the examples except for D, we compared the errors between the original and ROBERT results on the validation/test set. In example D, the original publication provided a database with an external test set, and we evaluated errors on the same test set by using the "--csv_test D_test.csv" option. Example E also included a third set (test) but we could not find information on the datapoints used for that test and, therefore, we compared the error obtained in ROBERT's validation set. In the case of example F, involving a classification method, we employed the "--type clas" option. A summary of the command lines used for each example is detailed in Table S6.

Table S6. Command lines used for each example.

Example	Command line used
A	python -m robert --csv_name "A.csv" --y "G_Exp" --names "ligand"
B	python -m robert --csv_name "B.csv" --y "ddG" --names "LNiArCl"
C	python -m robert --csv_name "C.csv" --y "%ee" --names "Name" --auto_test "False"
D	python -m robert --csv_name "D.csv" --y "Spectral_Integral" --names "ID" --csv_test "D_test.csv" --auto_test "False"
E	python -m robert --csv_name "E.csv" --y "target_barrier" --names "id" --auto_test "False"
F	python -m robert --csv_name "F.csv" --y "Outcome" --names "Name" --type "clas" --auto_test "False"

By default, ROBERT generates results from two models: one comprising all descriptors that pass the data curation process and another containing only the most important descriptors chosen by the PFI filter. When evaluating ROBERT's performance against the original publication, our main criterion was to choose the

predictor with the highest ROBERT score (Table S7). In the cases where multiple models achieved the same score, we prioritized the one with the lowest error. For a reliable comparison, we used the scaled error, which involves dividing the error by the range of the target value, from its minimum to maximum. The error type selected for comparison, RMSE or MAE for examples A-E and error in accuracy for F, was the type reported by the authors. In cases where both RMSE and MAE were reported, we opted for a comparison using RMSE.

Table S7. Summary of the results from the benchmarking using examples A-F.

Example	Model	Type of Error	Error	Range	Scaled error (%)
A	PFI	MAE	0.72	11.5	6.3
B	PFI	RMSE	0.12	1.1	11.0
C	PFI	RMSE	6.5	99.9	6.5
D	No PFI	MAE (test)	1.9e+05	2.4e+06	7.8
E	No PFI	MAE	1.2	24.0	5.0
F	No PFI	ACC	0.13*	1.0	13.0

*Accuracy = 0.87

All the generated CSV databases and ROBERT reports can be accessed in the ROBERT_data.zip file available in the ESI or in our Zenodo repository[13] (in the Benchmark folder). The names of the folders and CSV files correspond to the letter of their example (A, B, etc.).

SMILES-to-Predictor Examples

In the two examples presented, we used the AQME module to generate molecular and atomic descriptors. This module requires AQME[8] and xTB,[14] which can be installed with the following command lines (the versions used of each program can be found in the corresponding report PDF files):

```
conda install -c conda-forge aqme
```

```
conda install -c conda-forge xtb
```

In example A, we downloaded the ESOL database[15] and kept only three columns, for compound names (renamed as code_name), SMILES strings, and solubility (y values). To improve the accuracy of the xTB-generated descriptors, we employed water as the solvent in xTB geometry optimizations and single point calculations with the " --qdescp_keywords "--qdescp_solvent h2o" " argument in the command line. To replicate this workflow, users should follow the steps shown in the reproducibility section of the ROBERT report, which involves running this command line:

```
python -m robert --csv_name "A.csv" --y "solubility" --aqme --qdescp_keywords "--qdescp_solvent h2o"
```

In the case of Example B,[16] we first downloaded the database from the corresponding GitHub repository (vaskas_features_properties_smiles_filenames.csv from https://github.com/pascalfriederich/vaskas-space/blob/master/data). Subsequently, we filtered the dataset to keep only three columns: the compound names (renamed as code_name), SMILES strings, and barrier (y values). The original SMILES column contained hydrogenated versions of the Ir catalysts. To simplify conformer generation and descriptor generation, we transformed the complexes into the initial Ir square planar complexes by removing the "[H]" suffix from the SMILES strings.

As the target Ir catalysts exhibit a square planar geometry, we added a new column labeled "complex_type" in the CSV database to specify this geometry during the geometry optimization process using AQME. We also included two other columns, "charge" and "mult", set to 0 for charge and 1 for multiplicity across all cases. Furthermore, a column titled "geom" was introduced, employing the "Ir_squareplanar" rule to ensure the accurate distribution of ligands as described in the original paper (i.e., maintaining the *trans* arrangement of the two type "A" ligands).[16] In this example, ROBERT incorporated the atomic properties of the Ir metal center using SMARTS matching with the " --qdescp_keywords "--qdescp_atoms [Ir]" " argument in the command line. To replicate this workflow, users should follow the instructions of the reproducibility section in the ROBERT PDF report, which conclude with the execution of the following command line:

```
python -m robert --csv_name "B.csv" --y "barrier" --aqme --qdescp_keywords "--qdescp_atoms ['Ir']"
```

All the details of the steps performed in examples A and B are available in the “Examples / Full workflow from SMILES” and “Examples / SMILES workflow with atomic properties” sections, respectively, of our online documentation.[1] The CSV databases and ROBERT reports for both examples are available in the ROBERT_data.zip file from the ESI or our Zenodo repository[13] in the SMILES_workflows folder). The names of the folders and CSV files correspond to the letter of their example (A and B).

Discovery of Pd Luminescent Probes

In this example, we compiled all previously published oxazolone-Pd complexes with available quantum yield (QY) data,[17–20] resulting in a total of 20 complexes. These compounds were depicted using ChemDraw, and their corresponding SMILES notations were then added to a CSV file, accompanied by their compound names (under the column “code_name”) and QYs. This database can be accessed in the ROBERT_data.zip file, available either through the ESI or our Zenodo repository[13] (Pd.csv within the 'Pd_workflow' folder).

The initial predictor with 20 entries led to a low ROBERT score of 5 (Figure S8, left). This score was influenced by an unbalanced database, with an insufficient number of data points in the high QY range (Figure S8, middle). To rectify this issue, three new compounds (**23**, **24**, and **25**) were synthesized, aiming to balance the dataset and establish a more reliable model. The selection of these additional molecules was based on their rapid synthetic availability, guided by logical human decisions. For instance, we anticipated that introducing a Me group in the *para* position of a Py ligand would sustain the high QY observed in complexes **1**, **2**, and **3**. The expanded dataset with 23 datapoints improved the ROBERT score to 7 (Figure S8, right).

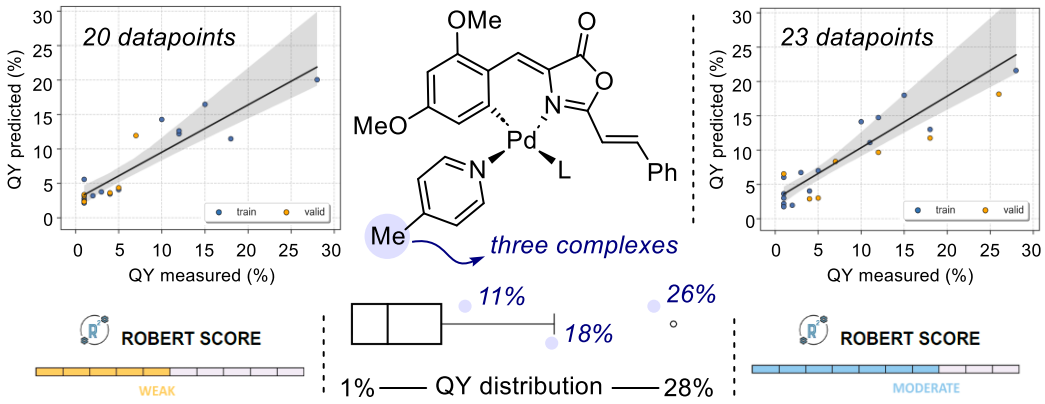


Figure S8. Left: ML predictor using the 20 previously published complexes. Middle: Three new synthesized complexes for data compensation. Right: ML predictor adding the three new complexes (23 datapoints). X

Figure S9 displays the 23 data points used in the predictor. Additionally, an external test set comprising 15 new candidates was created (Figure S10). These candidates were selected based on the availability of ligands and oxazolone cores within our chemical inventory.

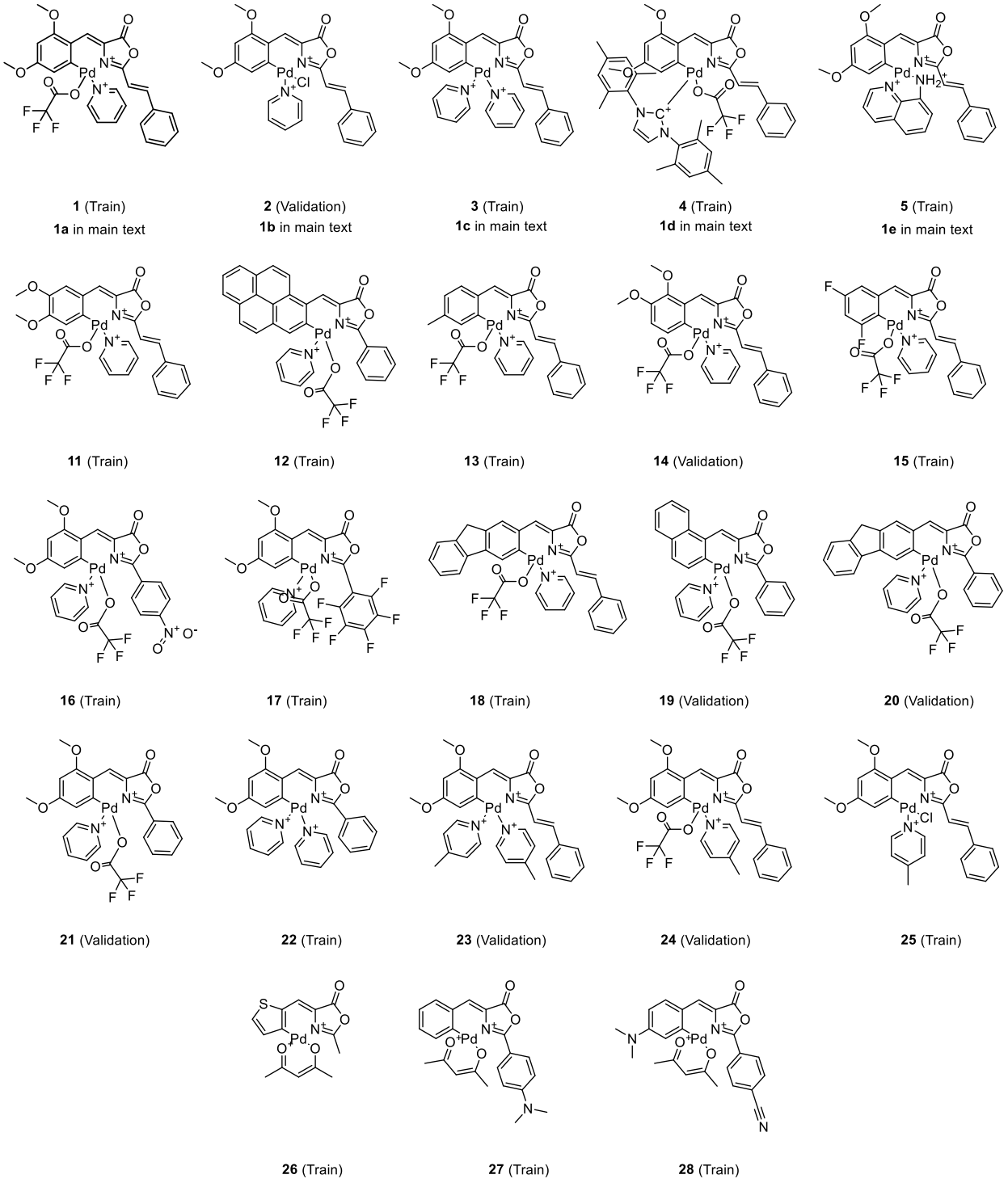


Figure S9. Compound structures used as database for predictive model generation.

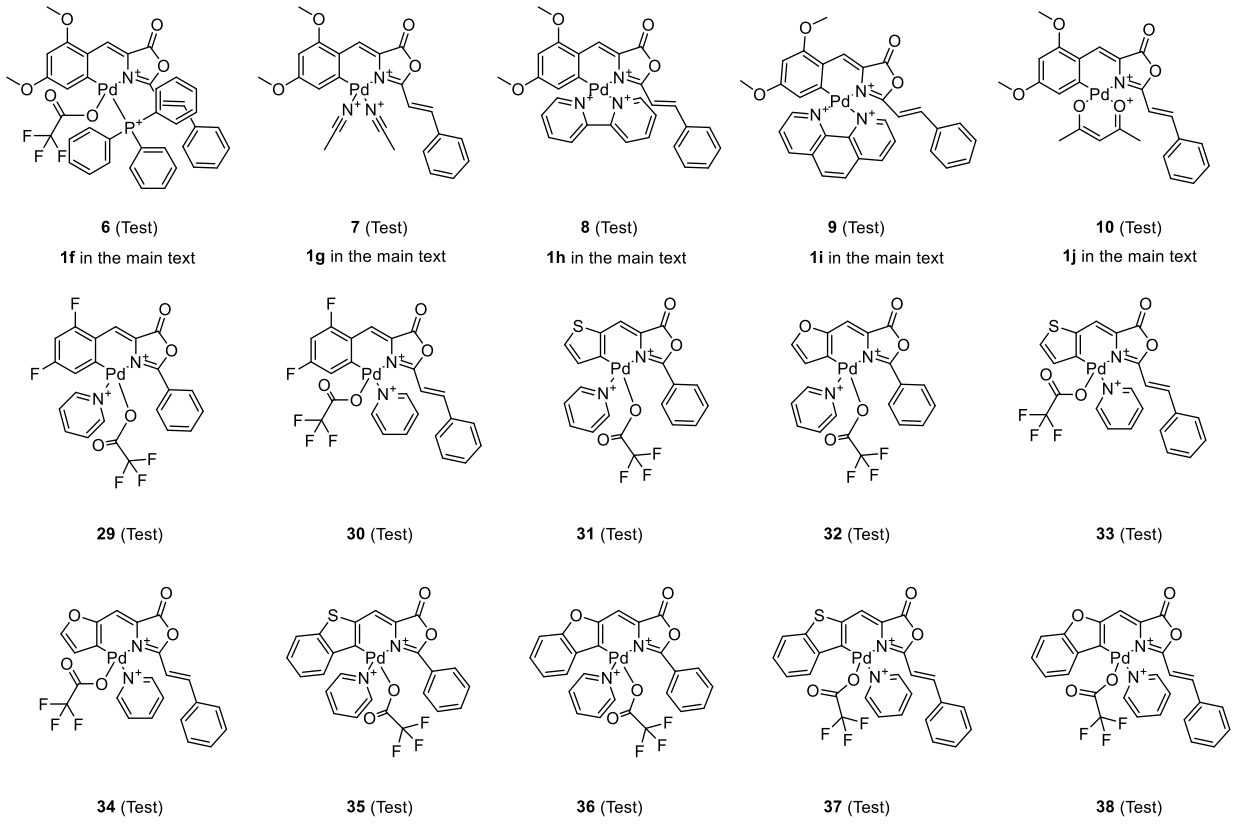


Figure S10. Compound structures used as external test set.

In this example, ROBERT incorporated the atomic properties of Pd and dichloromethane as the solvent during descriptor generation with the " --qdescp_keywords "--qdescp_atoms ['Pd'] --qdescp_solvent ch2cl2" " argument in the command line. As the target Pd complexes exhibit a square planar geometry, we added a new column labeled "complex_type" in the CSV database to specify this geometry during the geometry optimization process of AQME. We also included two other columns, "charge" and "mult", to define charges and multiplicities. Furthermore, a column titled "geom" was introduced, employing geometric rules to ensure the accurate distribution of ligands as observed in related crystal structures.[18,20] These rules keep only molecules with the N atom of the oxazolone *trans* to the N atom of the Py and 4-picoline ligands, the P atom of the PPh_3 ligand, and the C atom of the NHC ligand. In the CSV input files, the rules are defined using the atoms bonding to Pd, as N-Pd-X (X=N,C,P) atoms, at 180 degrees (defined generally as ['[N+][Pd][X+]',180] in the CSV input files). To replicate this workflow, users should follow the instructions of

the reproducibility section in the ROBERT PDF report, which conclude with the execution of the following command line:

```
python -m robert --csv_name "Pd.csv" --y "QY" --aqme --qdescp_keywords "--qdescp_atoms ['Pd'] --qdescp_solvent ch2cl2" --csv_test "Pd_test.csv"
```

The ROBERT job was repeated three times, showing very similar model metrics and ROBERT scores in all cases (Table S8). The databases used (Pd.csv and Pd_test.csv) and the resulting ROBERT report can be accessed in the ROBERT_data.zip file available in the ESI or in our Zenodo repository[13] (in the Pd_workflow folder).

Table S8. Errors and ROBERT score obtained in the three runs of the Pd workflow.

Run	Error valid. set (RMSE)	ROBERT Score
1	4.5	7
2	4.5	7
3	4.5	6

Experimental methods

Solvents were obtained from commercial sources and were used without further purification. All reactions were performed without special precautions against air and moisture. Electrospray ionization (ESI+) mass spectra were recorded using Bruker Esquire3000 plus™ or Amazon Speed ion-trap mass spectrometers equipped with standard ESI sources. High-resolution mass spectra-ESI (HRMS-ESI) were recorded using either a Bruker MicroToF-Q™ system equipped with an API-ESI source and a Q-ToF mass analyzer, or a TIMS-TOF system, both allowing a maximum error in the measurement of 5 ppm. Acetonitrile was used as solvent. For all types of MS measurements, samples were introduced in a continuous flow of 0.2 mL/min and nitrogen served both as the nebulizer gas and the dry gas. The ^{1}H, ^{13}C, ^{31}P and ^{19}F NMR spectra of the isolated products were recorded in CD_2Cl_2 solutions at 25 °C (other conditions were specified) on Bruker

AV300 (δ in ppm, J in Hz) at ^{1}H operating frequency of 300.13 MHz. The ^{1}H and ^{13}C NMR spectra were referenced using the solvent signal as internal standard, while ^{19}F NMR and ^{31}P NMR spectra were referenced to $CFCl_3$ and 85% H_3PO_4 in D_2O respectively. Absorption spectra were measured on a Thermo Scientific Evolution 600 spectrophotometer. The steady-state excitation–emission spectra were measured on a Jobin-Yvon Horiba Fluorolog FL-3-11 spectrofluorimeter. All measurements were carried out at room temperature on solutions of 10-5M concentration using quartz cuvettes of 1 cm path length. The measurement of the quantum yield values (Φ_{PL}) was carried out using the absolute method on a Quantaurus-QY C11347 spectrometer.

Characterization of Pd Complexes

Characterization

General Synthesis and Characterization of complexes **6, 23-25.** The complexes **1-5, 7-22** and **26-28** were previously reported by our group[17–20] and were characterized by comparison of their NMR data with those previously published.

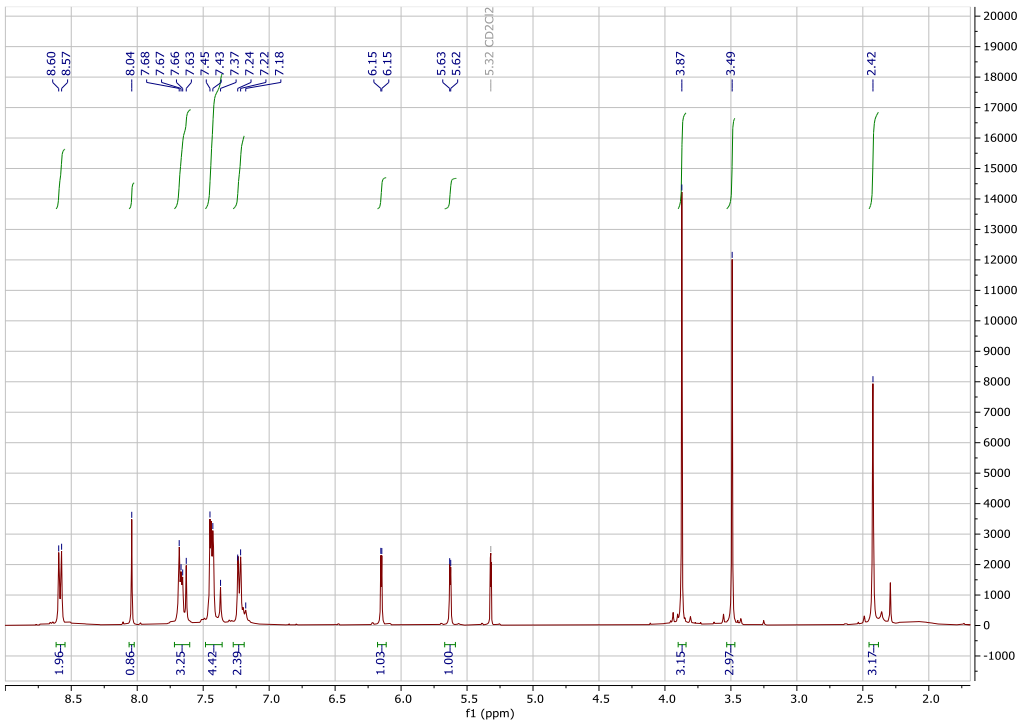
Synthesis of complex **6**. ^{1}H NMR (CD_2Cl_2, 300.13 MHz): δ 8.09 (1H), 7.68-6.94 (21H), 6.96 (1H), 6.11 (1H), 5.95 (1H), 3.83 (3H), 3.12 (3H). $^{13}C\{^{1}H\}$ NMR (CD_2Cl_2, 75.47 MHz): δ 163.6 (C_{ar}), 162.4 (C_{ar}), 160.8 (C_{ar}), 147.9 (C_{ar}), 145.4 (C_{ar}), 134.5 (C_{ar}), 134.3 (C_{ar}), 132.9 (C_{ar}), 131.4 (C_{ar}), 130.9 (C_{ar}), 130.8 (C_{ar}), 129.5 (C_{ar}), 128.9 (C_{ar}), 128.8 (C_{ar}), 128.5 (C_{ar}), 127.0 (C_{ar}), 120.4 (C_{ar}), 117.9 (C_{ar}), 111.6 (C_{ar}), 95.3 (C_{ar}), 55.8 (C_{al}), 54.8 (C_{al}). ^{19}F NMR (CD_2Cl_2, 282.40 MHz): δ –74.98 (CF_3COO). ^{31}P NMR (CD_2Cl_2, 121.44 MHz): δ 32.8 (^{1}P). HRMS (ESI) m/z: $[M - CF_3COO]^+$ Calcd for $[C_{38}H_{31}O_4PPdN]^+$ 702.1034. Found 702.1045.

Synthesis of complex **23**. ^{1}H NMR (CD_2Cl_2, 300.13 MHz): δ 8.77 (2H), 8.61 (2H), 8.11 (1H) 7.39-7.18 (11H), 6.17 (1H), 5.77 (1H), 3.89 (3H), 3.54 (3H), 2.38 (3H), 1.99 (3H). $^{13}C\{^{1}H\}$ NMR (CD_2Cl_2, 75.47 MHz): δ 163.1 (C_{ar}), 162.9 (C_{ar}), 161.6 (C_{ar}), 160.7 (C_{ar}), 153.0 (C_{ar}), 152.1 (C_{ar}), 152.0 (C_{ar}), 150.9 (C_{ar}), 150.6 (C_{ar}), 149.5 (C_{ar}), 133.2 (C_{ar}), 131.9 (C_{ar}), 129.2 (C_{ar}), 128.9 (C_{ar}), 127.8 (C_{ar}), 127.4 (C_{ar}), 126.6 (C_{ar}), 119.4 (C_{ar}), 116.3 (C_{ar}), 109.6 (C_{ar}), 94.9 (C_{ar}), 56.0 (C_{al}), 55.7 (C_{al}), 21.3 (C_{al}), 20.8 (C_{al}). HRMS (ESI) m/z: $[M - C_6H_7N]^+$ Calcd for $[C_{26}H_{23}O_4PdN_2]^+$ 533.0697. Found 533.0698.

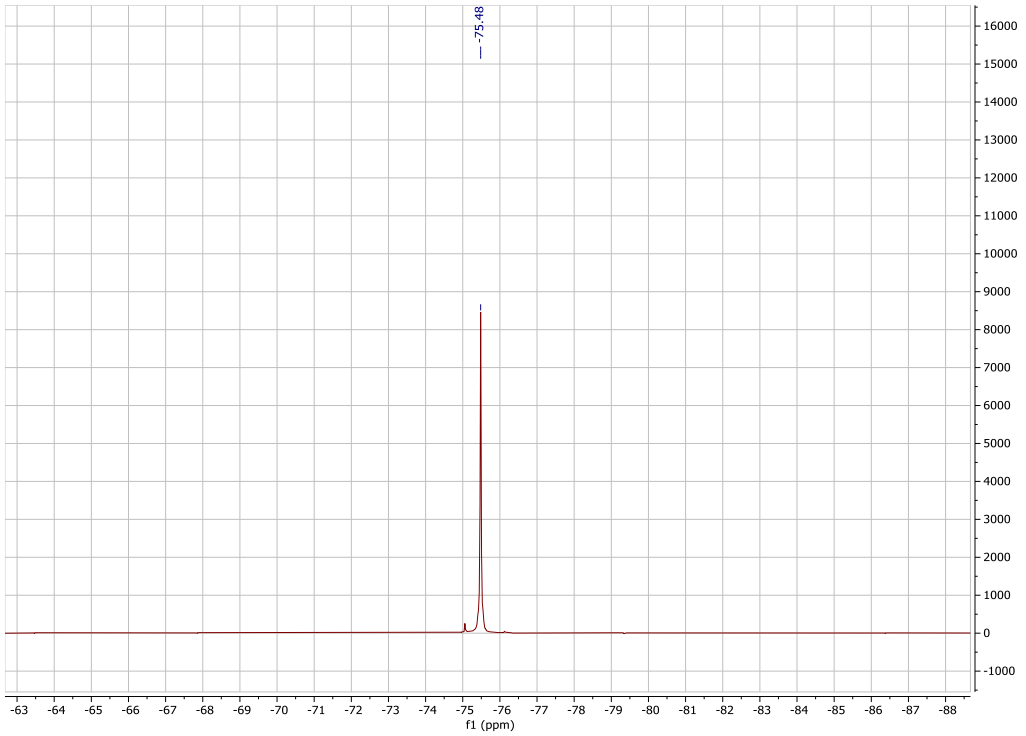
Synthesis of complex **24**. ^{1}H NMR (CD_2Cl_2, 300.13 MHz): δ 8.58 (2H), 8.04 (1H) 7.68-7.63 (3H), 7.45-7.37 (4H), 7.24-7.18 (2H), 6.15 (1H), 5.62 (1H), 3.87 (3H), 3.49 (3H), 2.42 (3H). ^{13}C{^{1}H} NMR (CD_2Cl_2, 75.47 MHz): δ 164.1 (C_{ar}), 162.5 (C_{ar}), 162.2 (C_{ar}), 160.4 (C_{ar}), 152.3 (C_{ar}), 151.5 (C_{ar}), 147.9 (C_{ar}), 145.7 (C_{ar}), 134.9 (C_{ar}), 132.5 (C_{ar}), 131.6 (C_{ar}), 129.4 (C_{ar}), 129.3 (C_{ar}), 126.6 (C_{ar}), 120.6 (C_{ar}), 116.9 (C_{ar}), 116.1 (C_{ar}), 112.3 (C_{ar}), 95.1 (C_{ar}), 56.2 (C_{al}), 55.5 (C_{al}), 21.4 (C_{al}). ^{19}F NMR (CD_2Cl_2, 282.40 MHz): δ –75.48 (CF_3COO). HRMS (ESI) m/z: $[M - CF_3COO]^+$ Calcd for $[C_{26}H_{23}O_4PdN_2]^+$ 533.0697. Found 533.0519.

Synthesis of complex **25**. ^{1}H NMR (CD_2Cl_2, 300.13 MHz): δ 8.60 (2H), 8.06-8.01 (2H), 7.69-7.55 (3H), 7.45-7.43 (3H), 7.21 (2H), 6.12 (1H), 5.62 (1H), 3.86 (3H), 3.46 (3H), 2.42 (3H). ^{13}C{^{1}H} NMR (CD_2Cl_2, 75.47 MHz): δ 164.8 (C_{ar}), 162.9 (C_{ar}), 162.4 (C_{ar}), 160.1 (C_{ar}), 152.9 (C_{ar}), 152.1 (C_{ar}), 151.0 (C_{ar}), 143.0 (C_{ar}), 135.6 (C_{ar}), 132.8 (C_{ar}), 131.2 (C_{ar}), 129.5 (C_{ar}), 129.1 (C_{ar}), 126.4 (C_{ar}), 120.9 (C_{ar}), 116.9 (C_{ar}), 115.5 (C_{ar}), 115.3 (C_{ar}), 95.1 (C_{ar}), 56.2 (C_{al}), 55.5 (C_{al}), 21.4 (C_{al}). HRMS (ESI) m/z: $[M - Cl]^+$ Calcd for $[C_{26}H_{23}O_4PdN_2Cl]^+$ 533.0697. Found 533.0458.

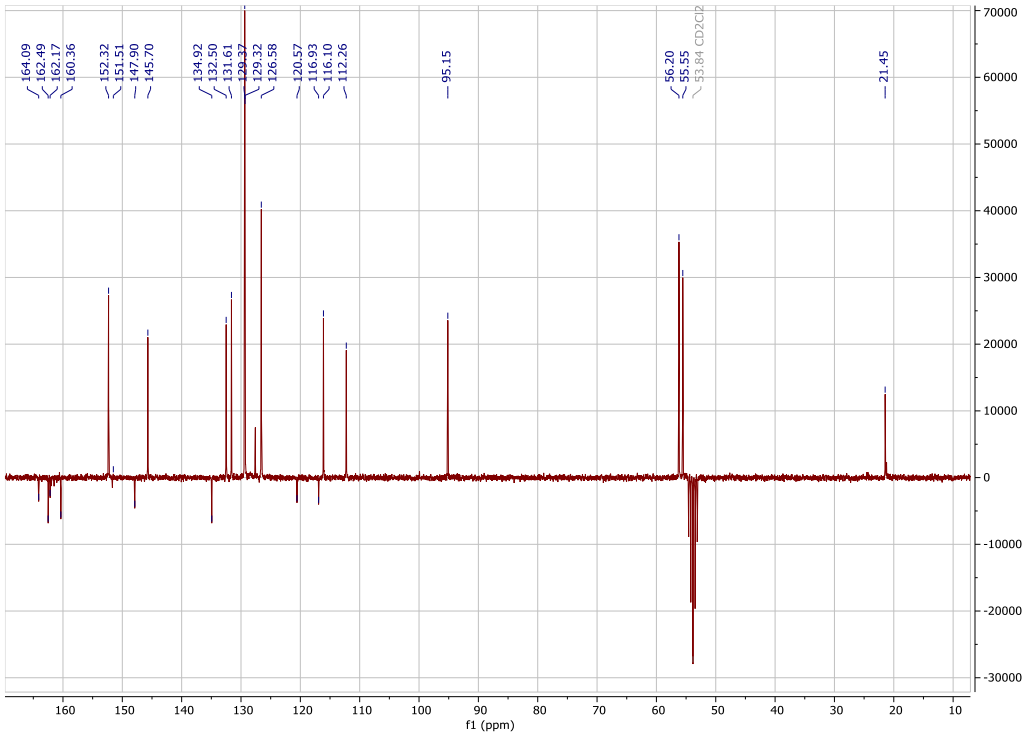
<u>NMR spectra</u>



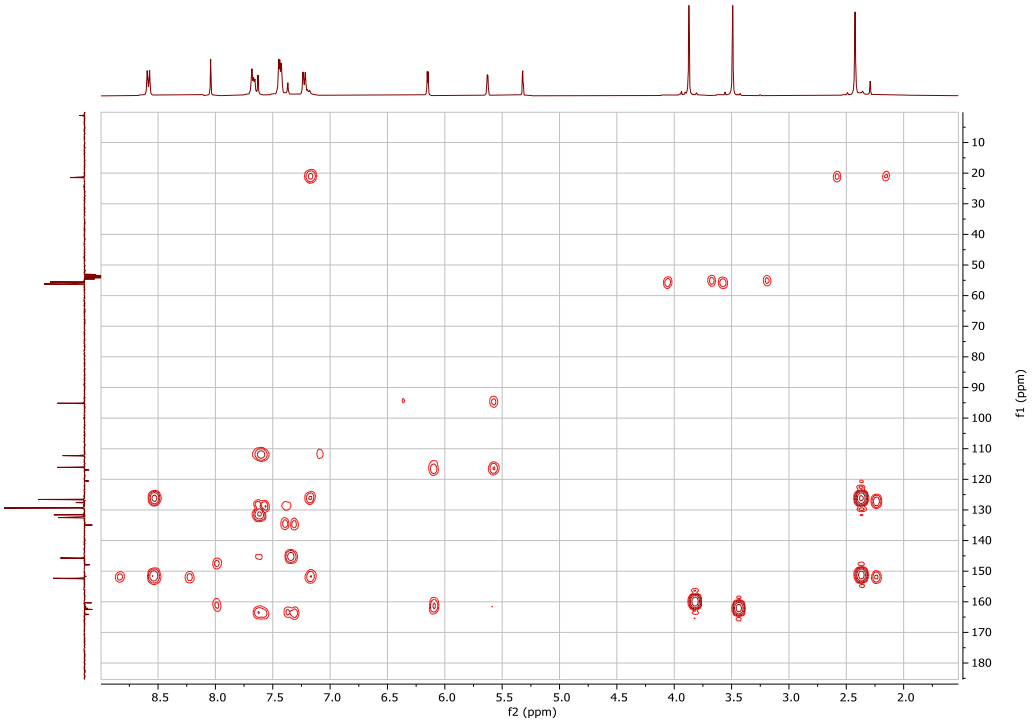
^{1}H-NMR spectrum (CD_2Cl_2, 300.13 MHz) of complex **24**



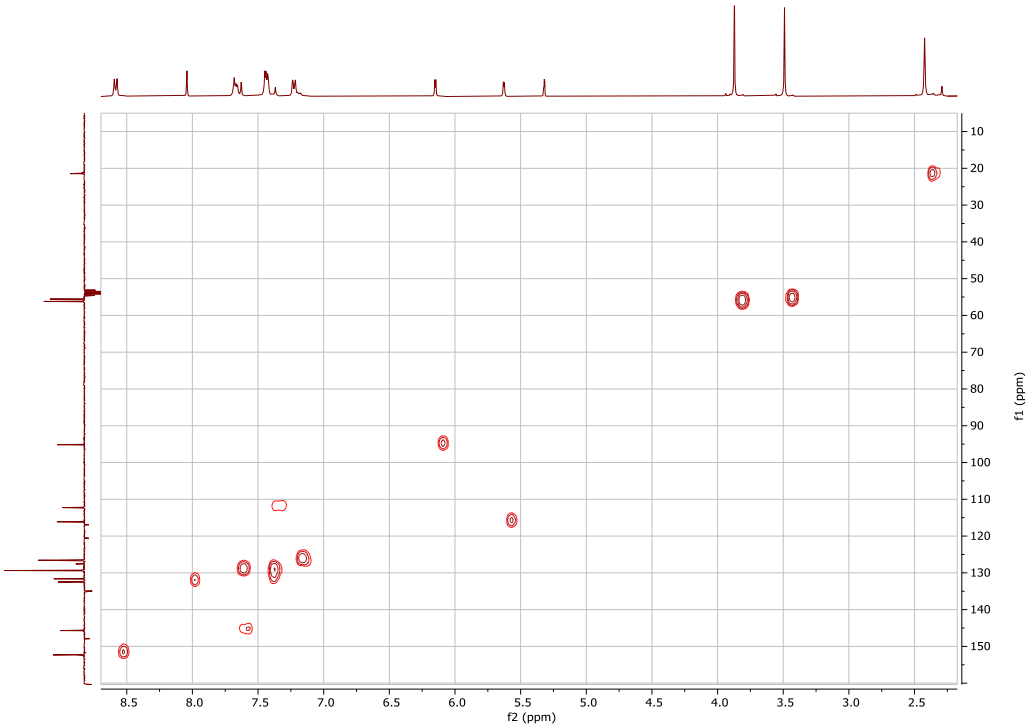
^{19}F-NMR spectrum (CD_2Cl_2, 282.40 MHz) of complex **24**



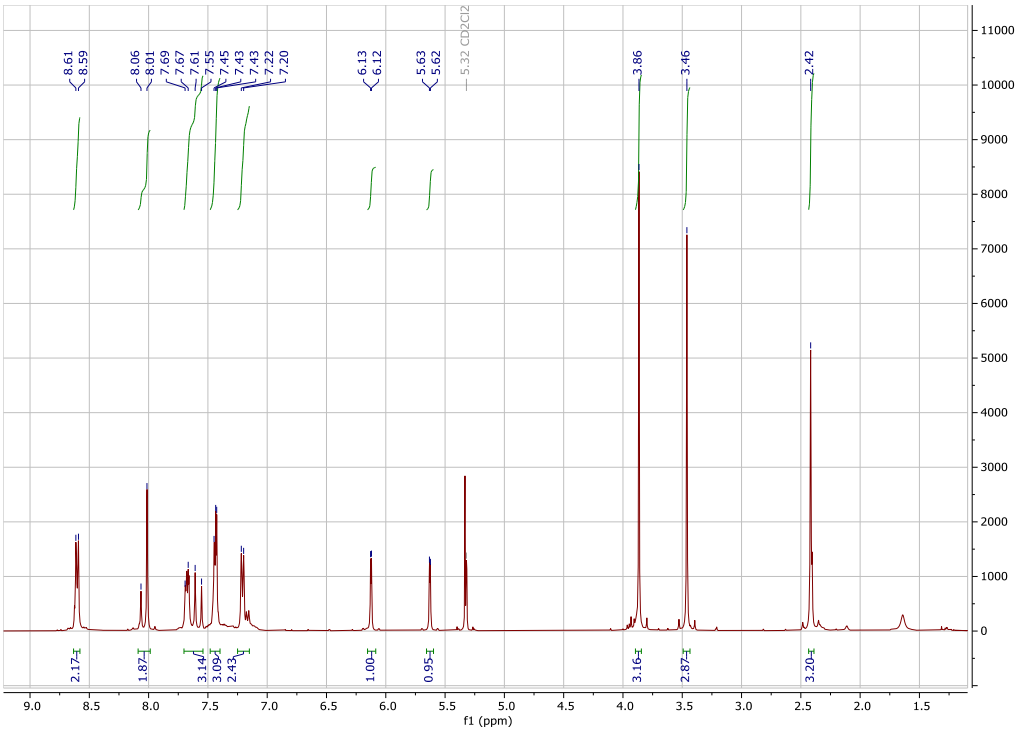
^{13}C{^{1}H} (APT) NMR spectrum (CD_2Cl_2, 75.47 MHz) of complex **24**



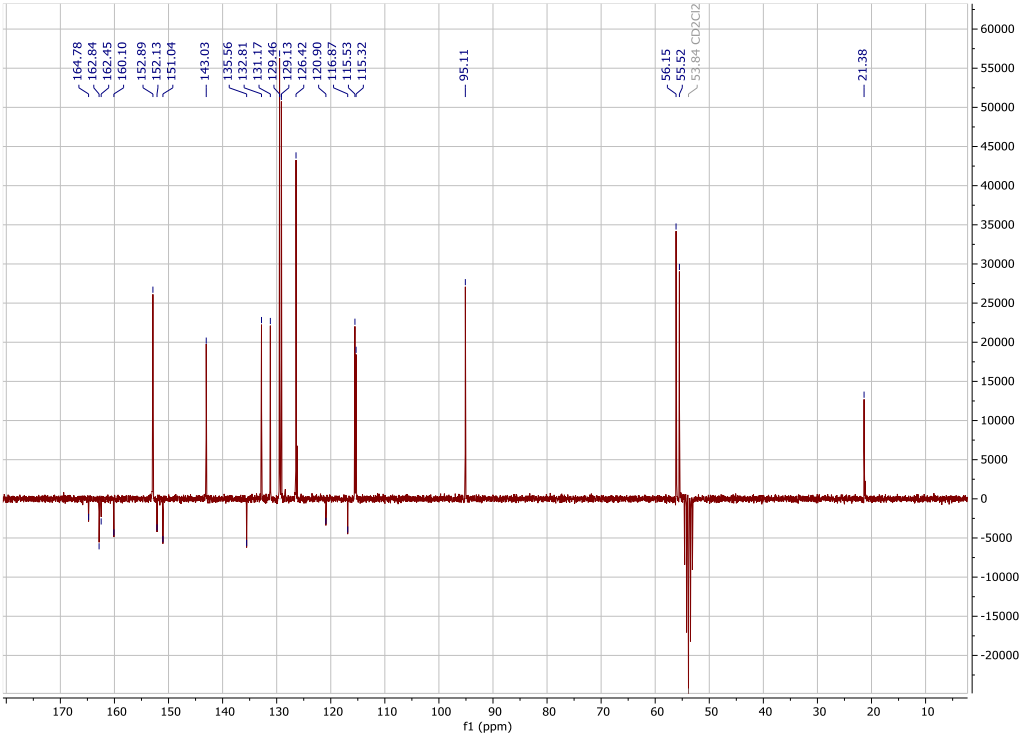
^{1}H - ^{13}C HSQC correlation spectrum of complex **24**



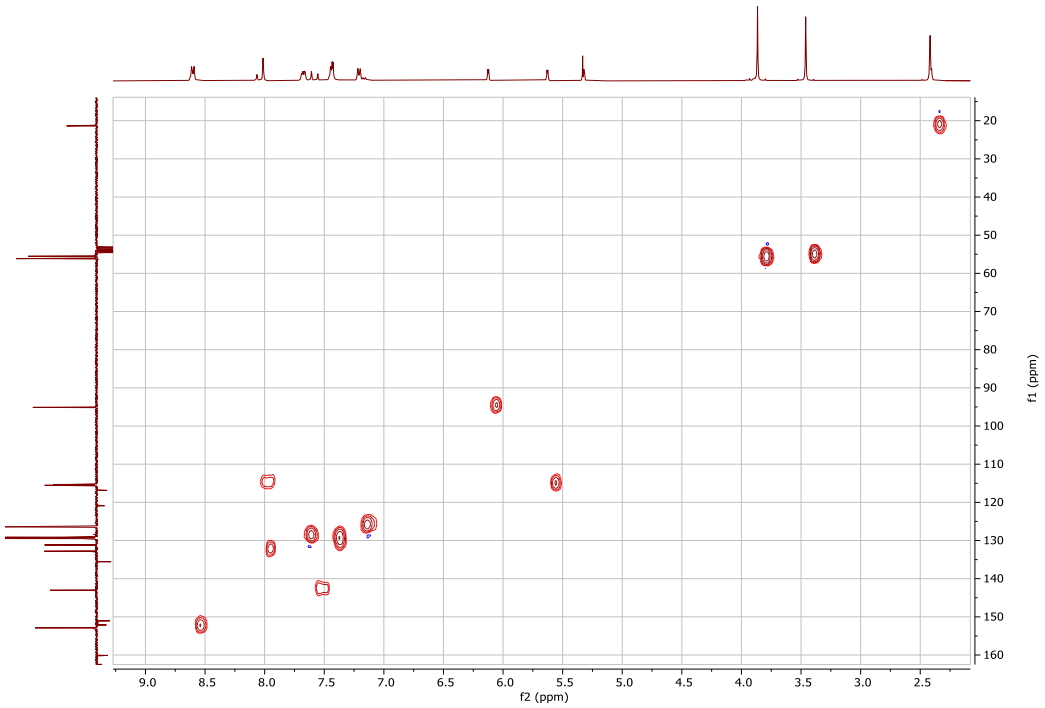
^{1}H - ^{13}C HMBC correlation spectrum of complex **24**



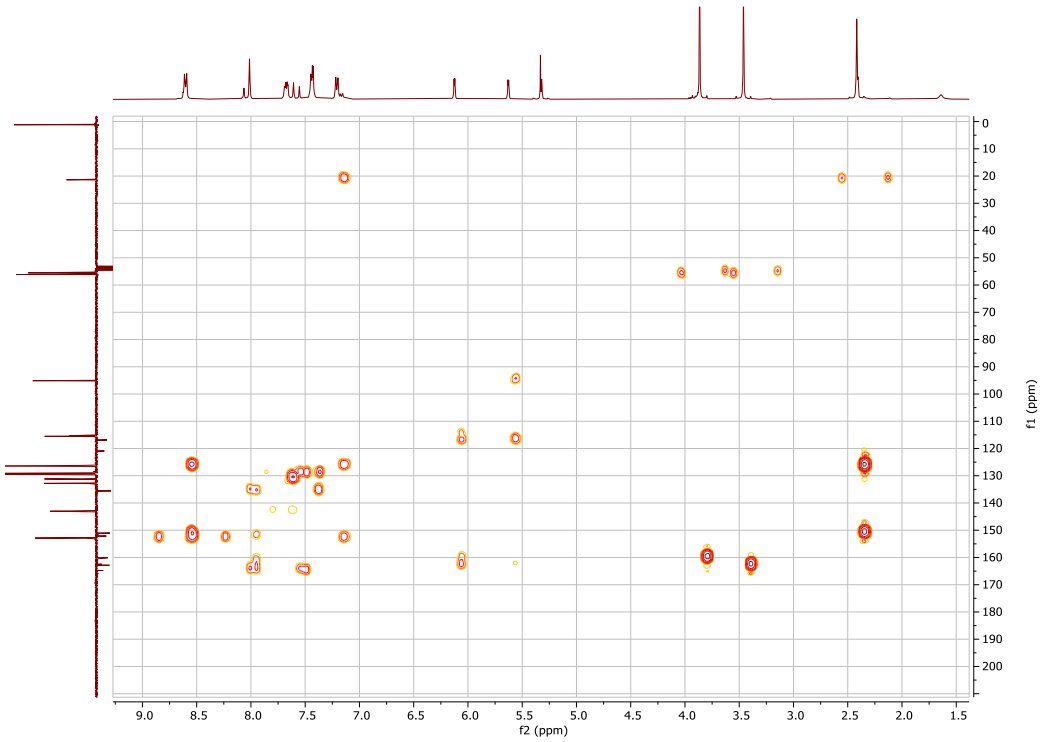
^{1}H-NMR spectrum (CD_2Cl_2, 300.13 MHz) of complex **25**



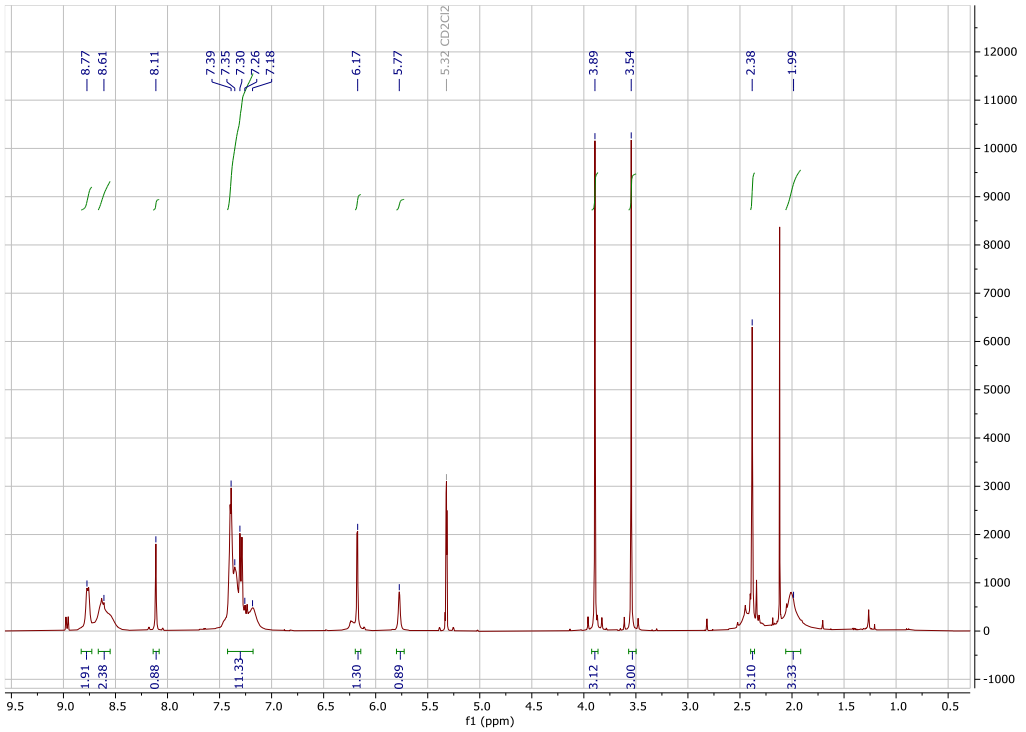
^{13}C{^{1}H} (APT) NMR spectrum (CD_2Cl_2, 75.47 MHz) of complex **25**



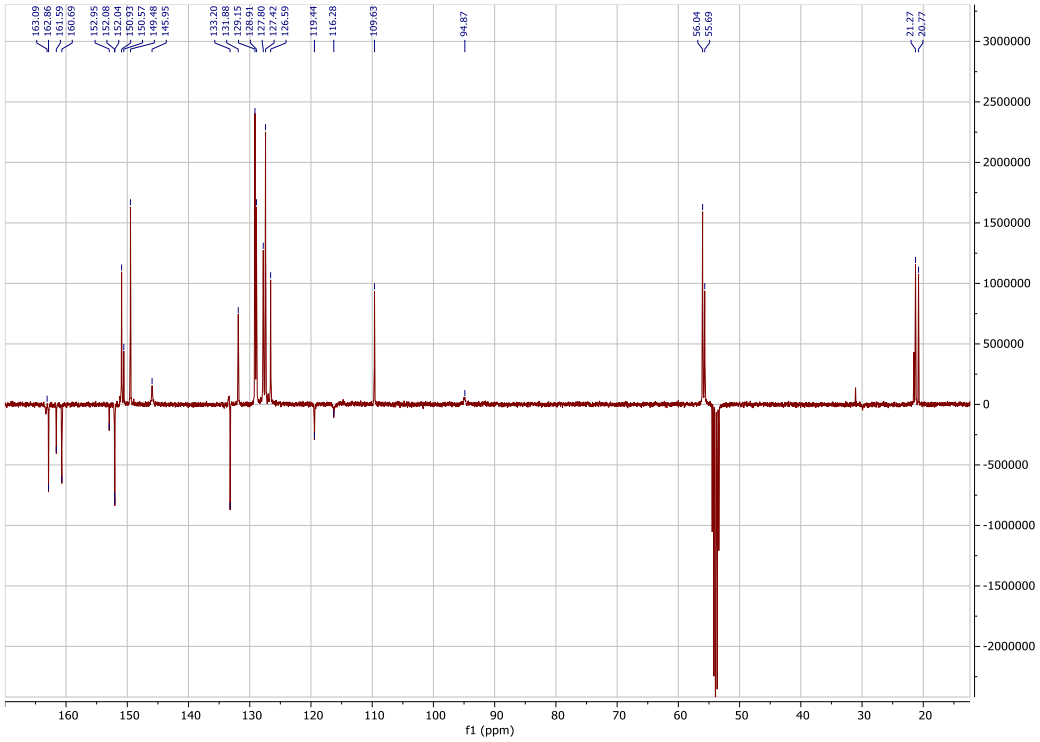
^{1}H - ^{13}C HSQC correlation spectrum of complex **25**



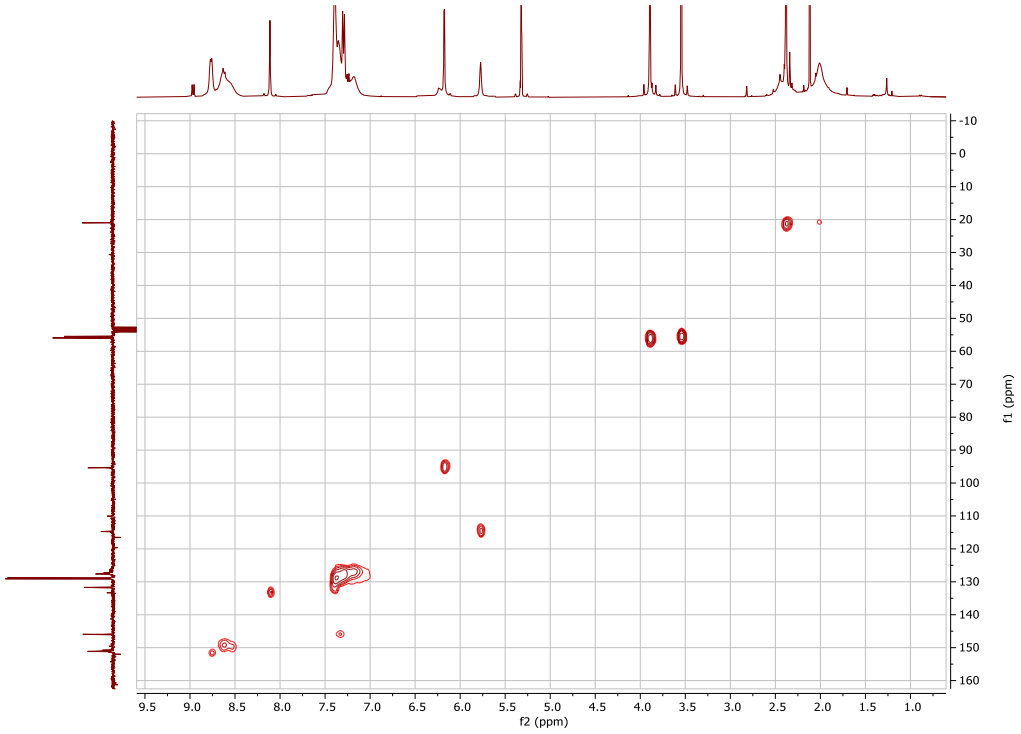
^{1}H - ^{13}C HMBC correlation spectrum of complex **25**



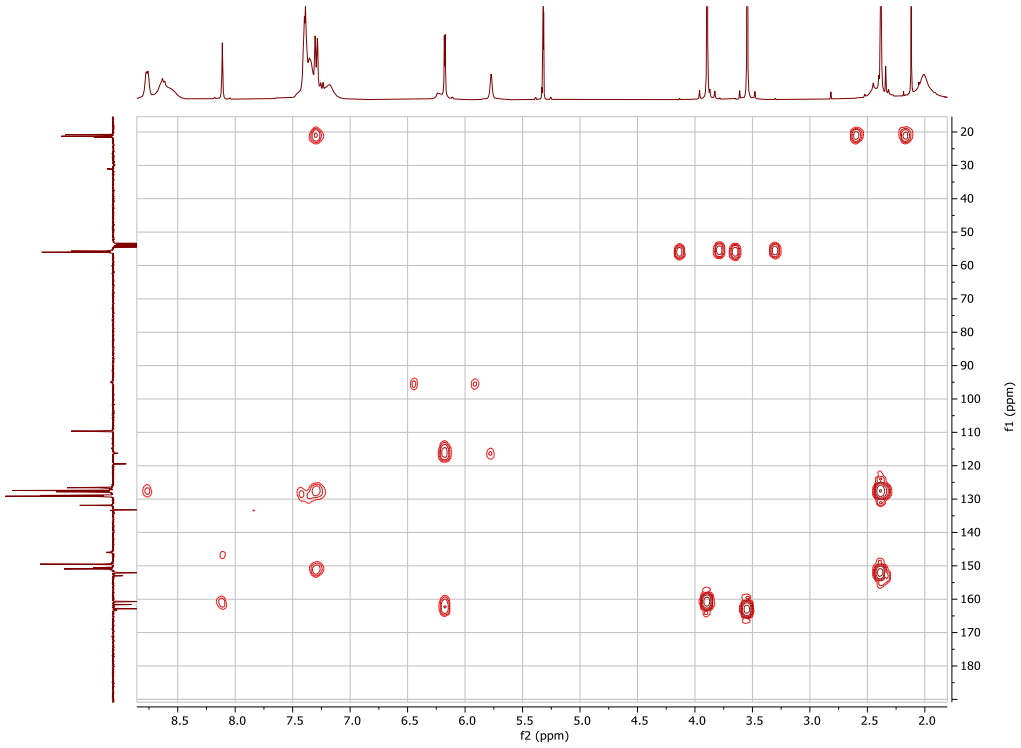
^{1}H-NMR spectrum (CD_2Cl_2, 300.13 MHz) of complex **23**



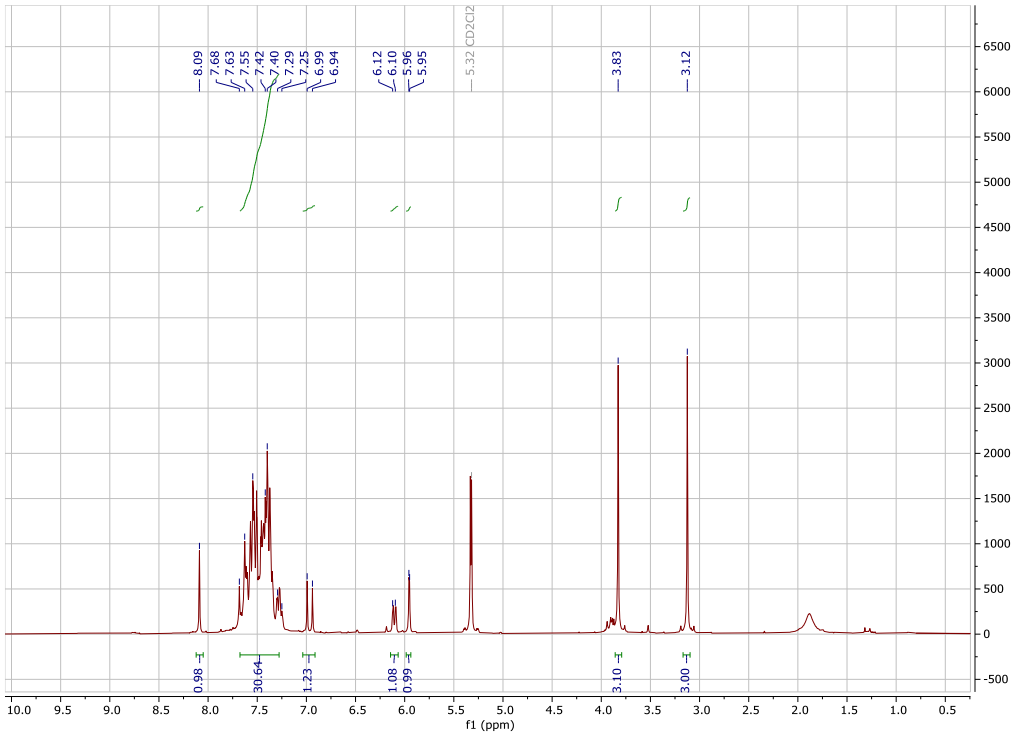
^{13}C{^{1}H} (APT) NMR spectrum (CD_2Cl_2, 75.47 MHz) of complex **23** at 243K



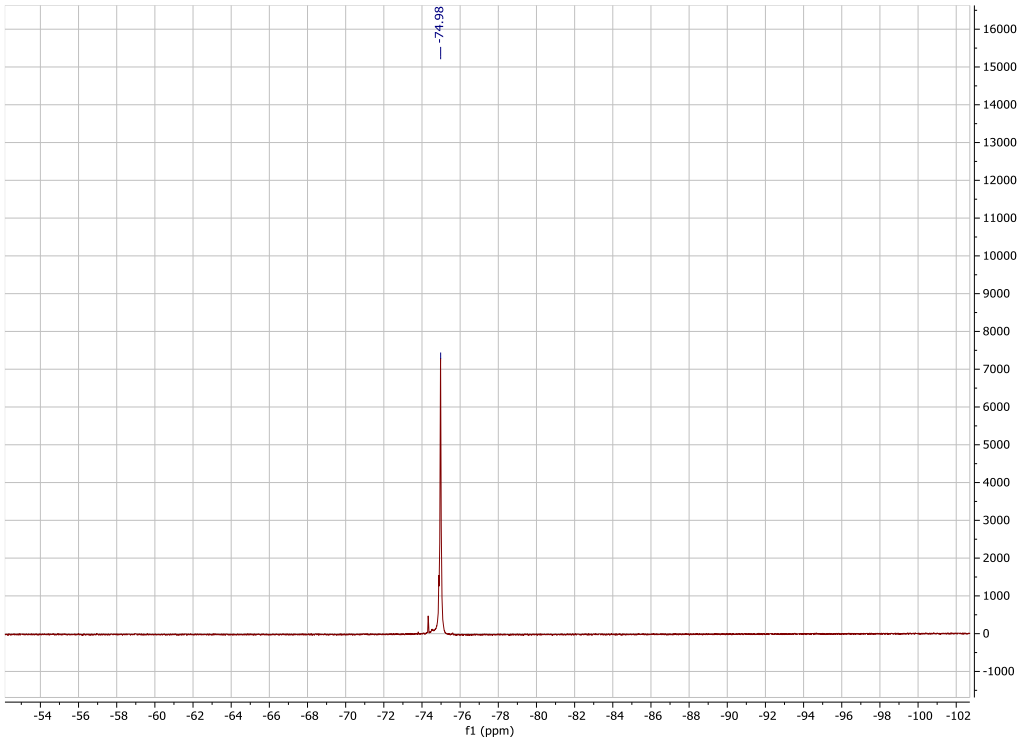
^{1}H - ^{13}C HSQC correlation spectrum of complex **23**



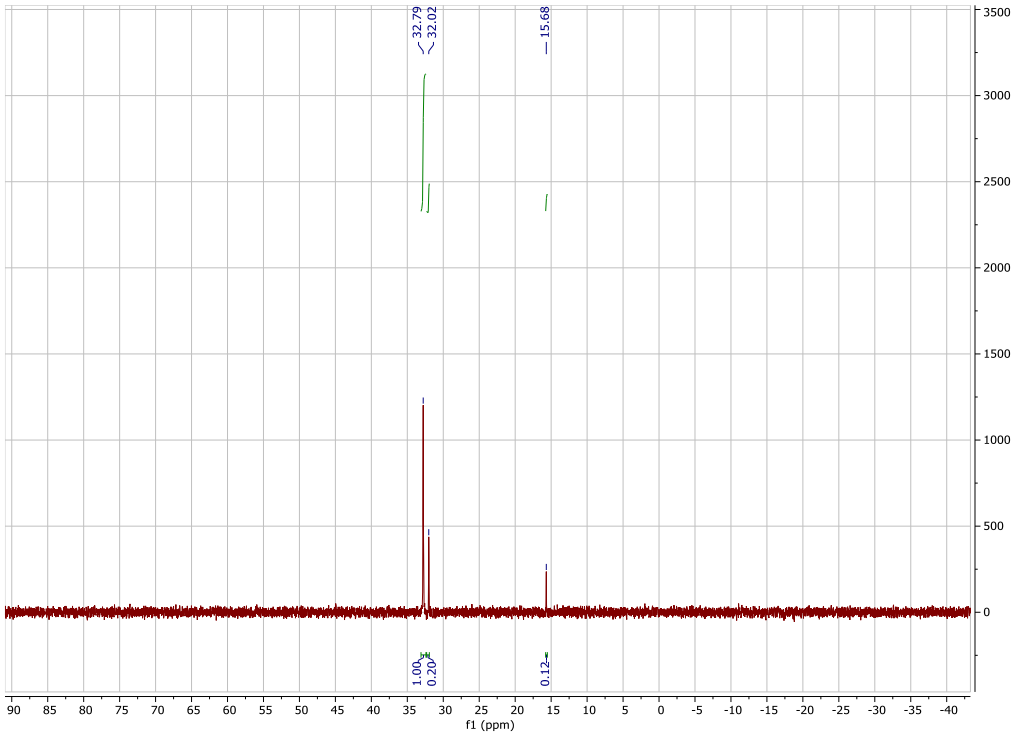
^{1}H - ^{13}C HMBC correlation spectrum of complex **23**



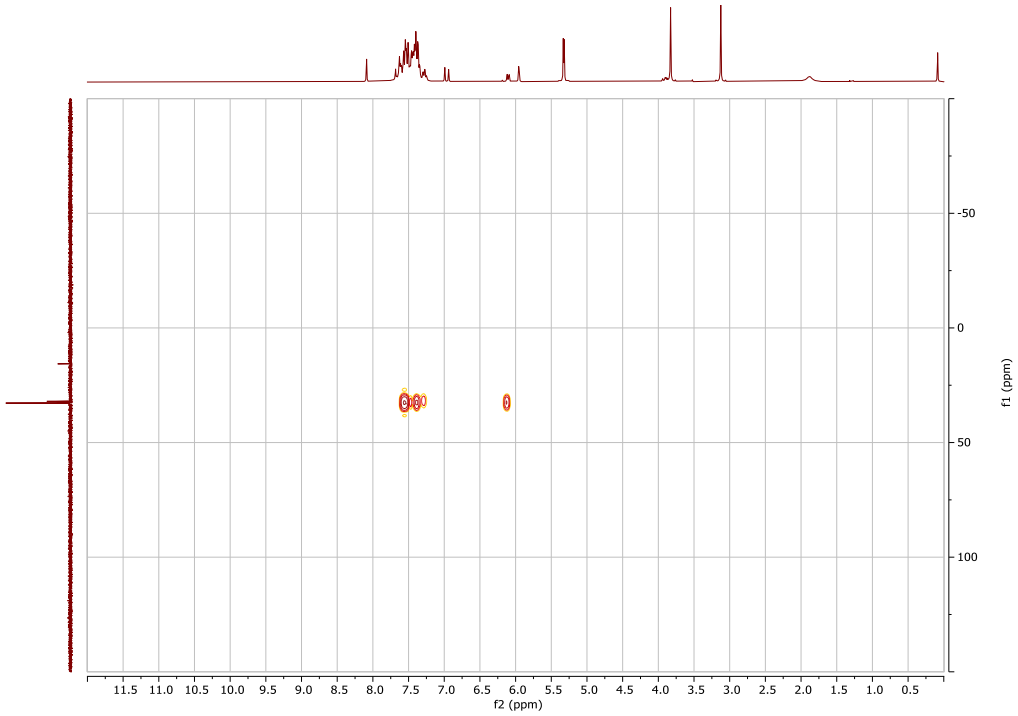
^{1}H-NMR spectrum (CD_2Cl_2, 300.13 MHz) of complex **6**



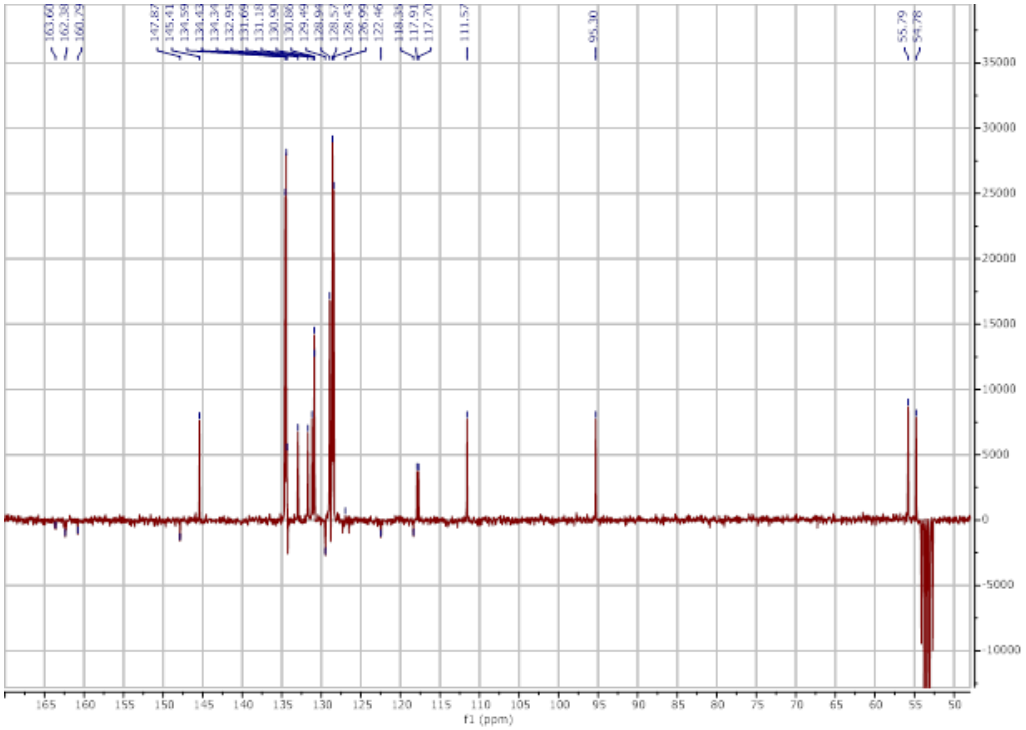
^{19}F-NMR spectrum (CD_2Cl2, 282.40 MHz) of complex **6**



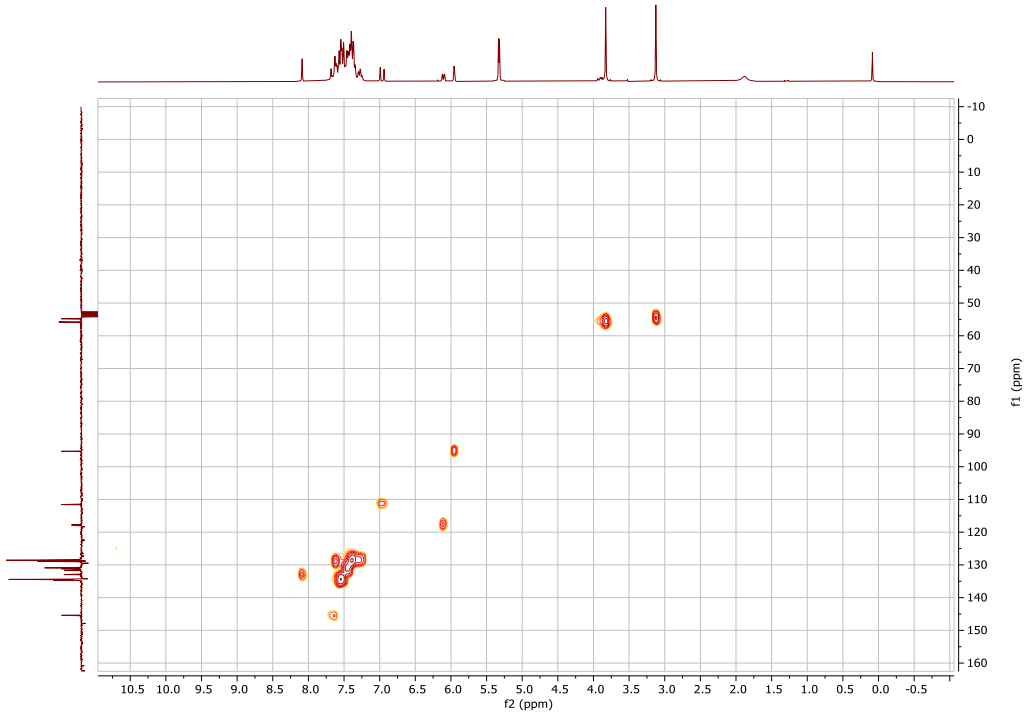
^{31}P-NMR spectrum (CD_2Cl_2, 121.44 MHz) of complex **6**



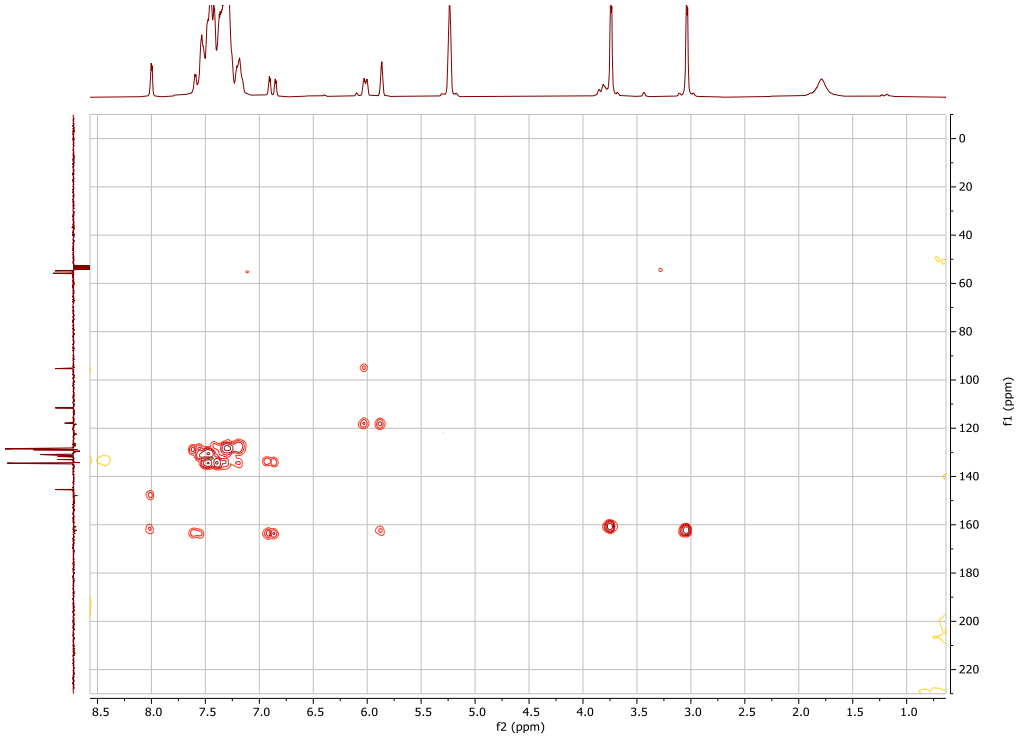
^{1}H – ^{31}P correlation spectrum of complex **6**



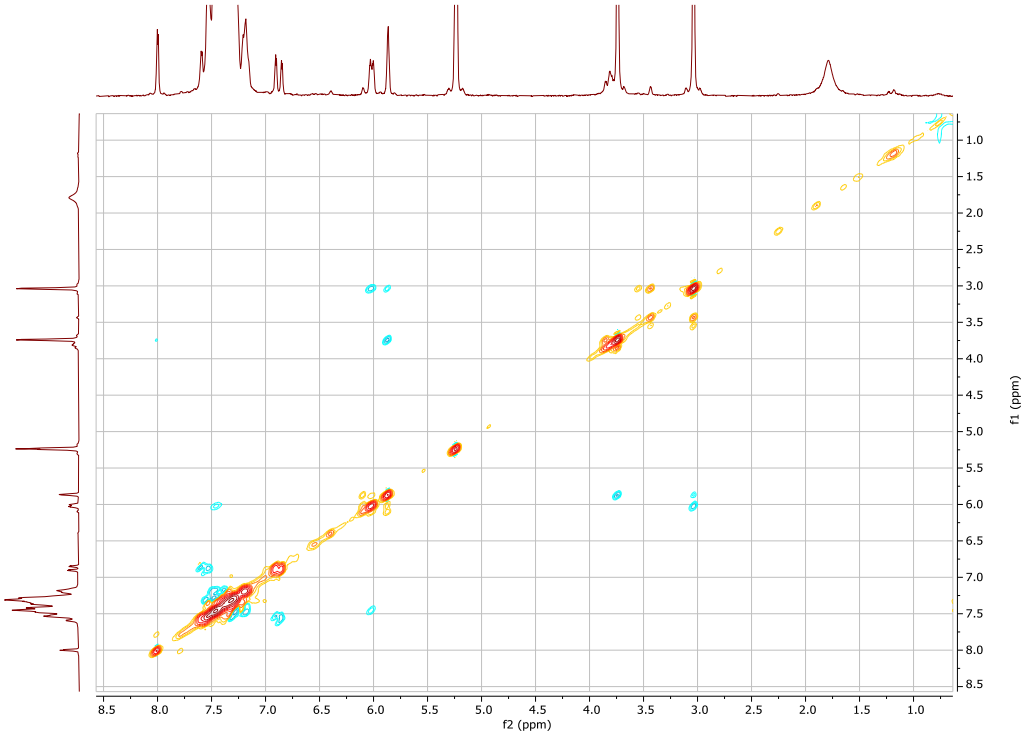
$^{13}C\{^{1}H\}$ (APT) NMR spectrum (CD_2Cl_2, 75.47 MHz) of complex **6**



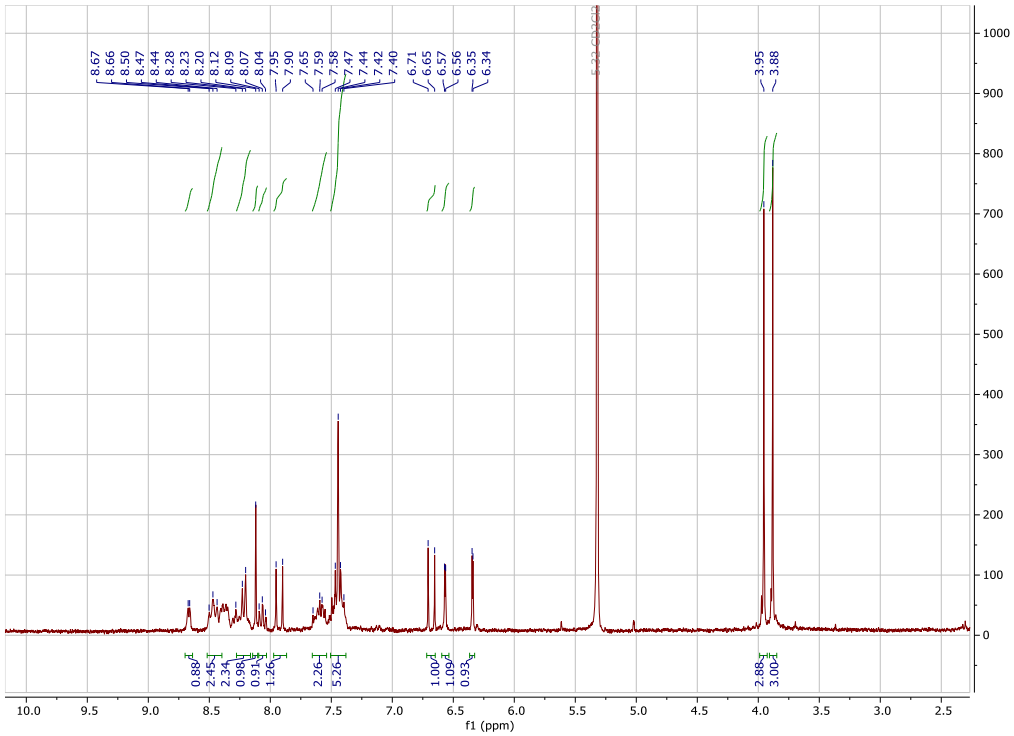
^{1}H - ^{13}C HSQC correlation spectrum of complex **6**



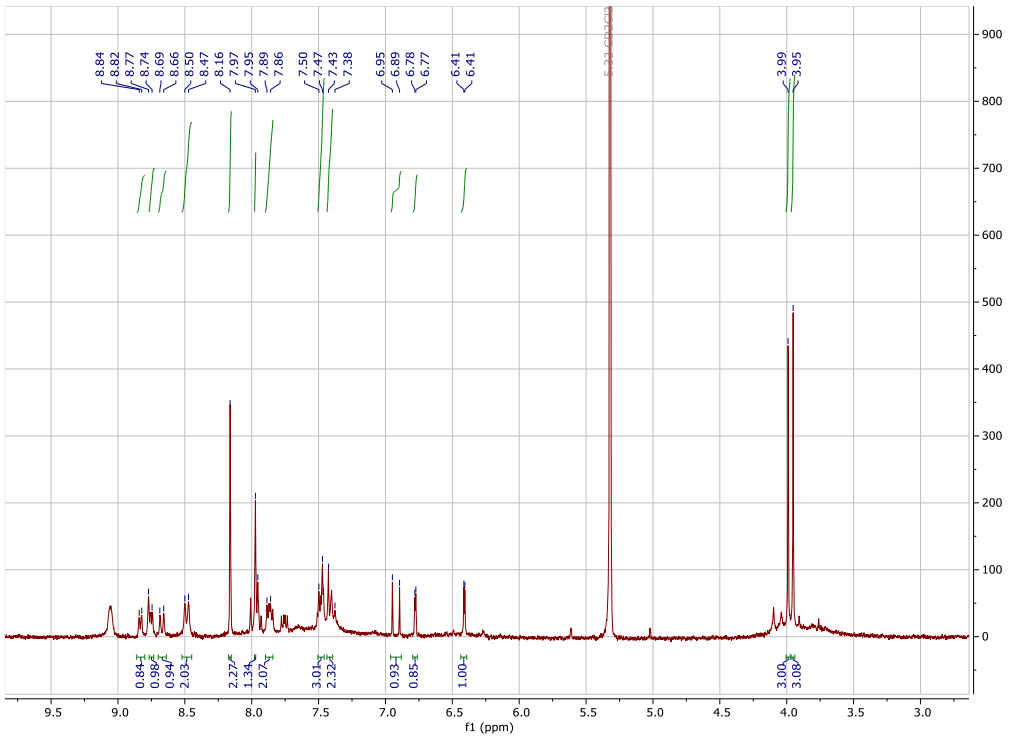
^{1}H - ^{13}C HMBC correlation spectrum of complex **6**



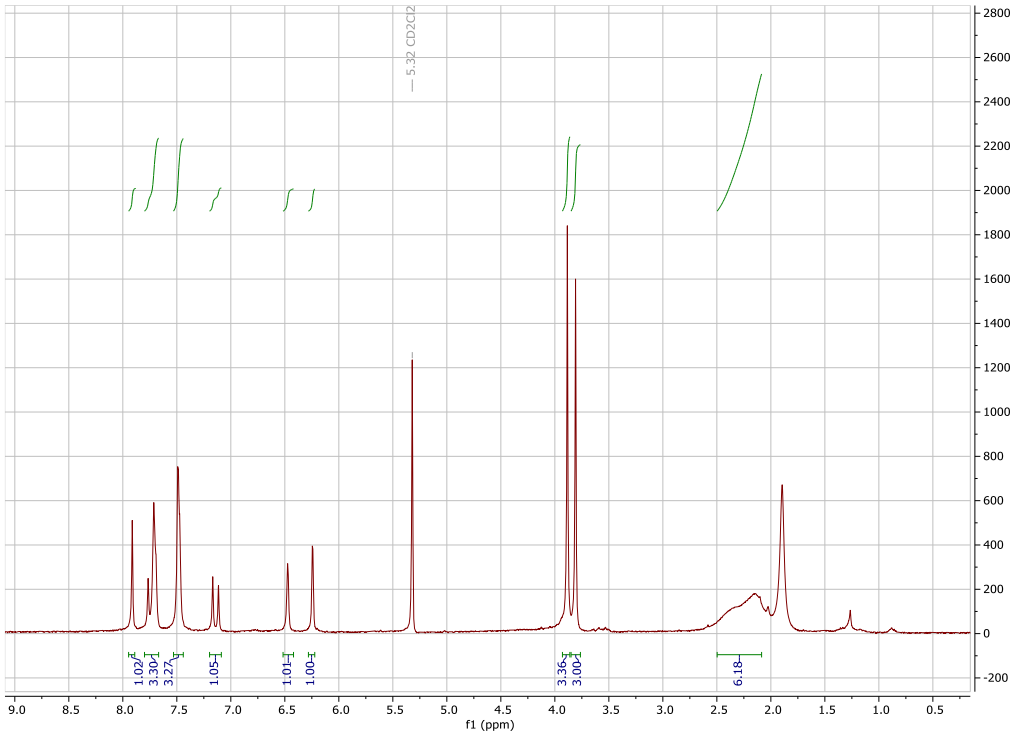
NOE spectrum of complex **6**



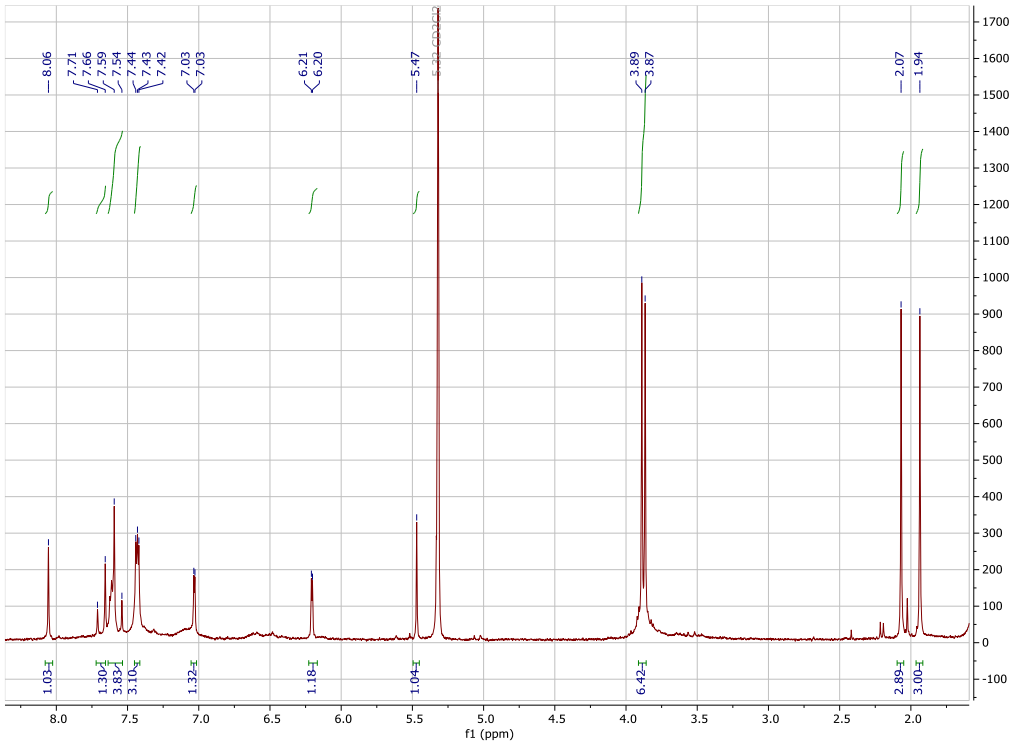
^{1}H-NMR spectrum (CD_2Cl_2, 300.13 MHz) of complex **8**



^{1}H-NMR spectrum (CD_2Cl_2, 300.13 MHz) of complex **9**



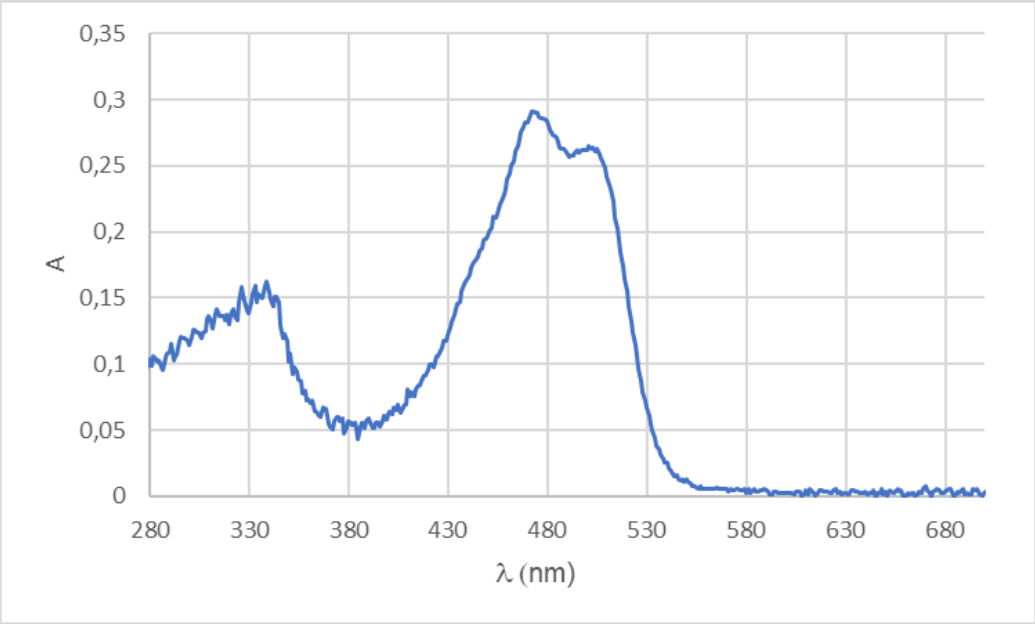
^{1}H-NMR spectrum (CD_2Cl_2, 300.13 MHz) of complex **7**



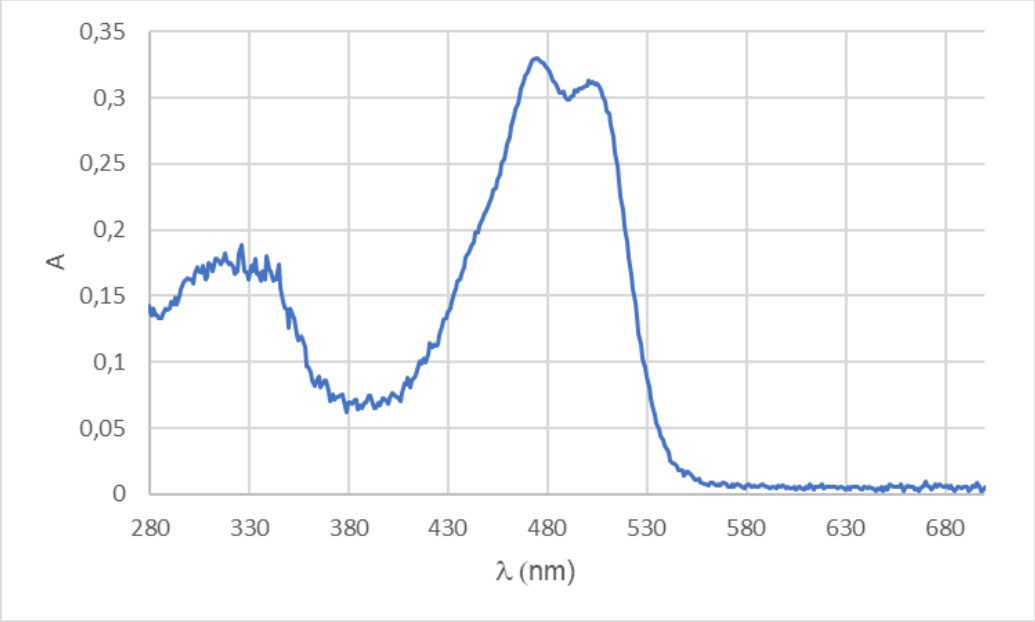
^{1}H-NMR spectrum (CD_2Cl_2, 300.13 MHz) of complex **10**

UV-Vis spectra

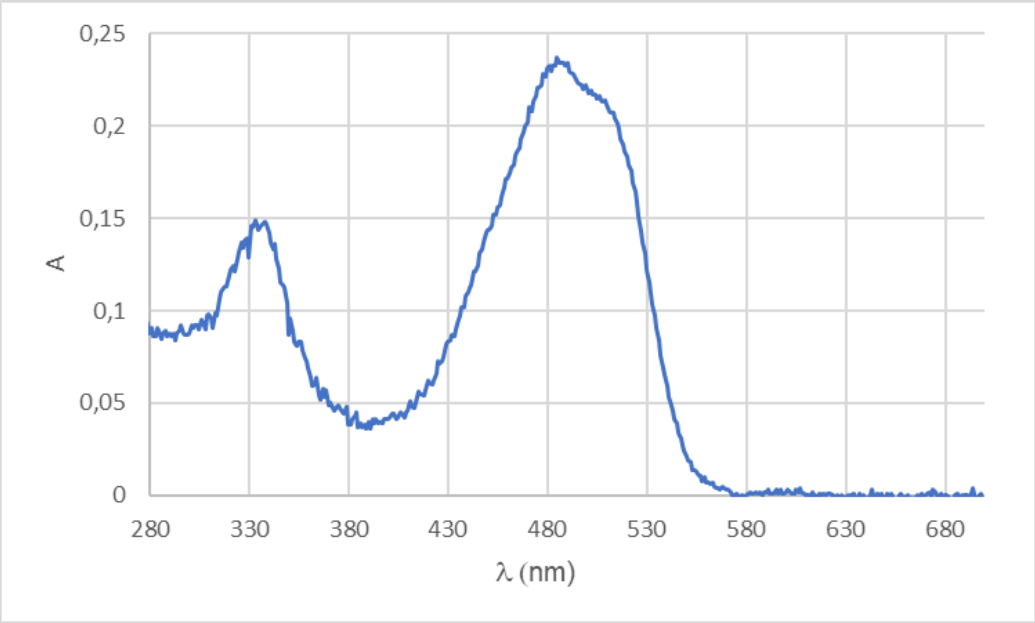
Complex **24**



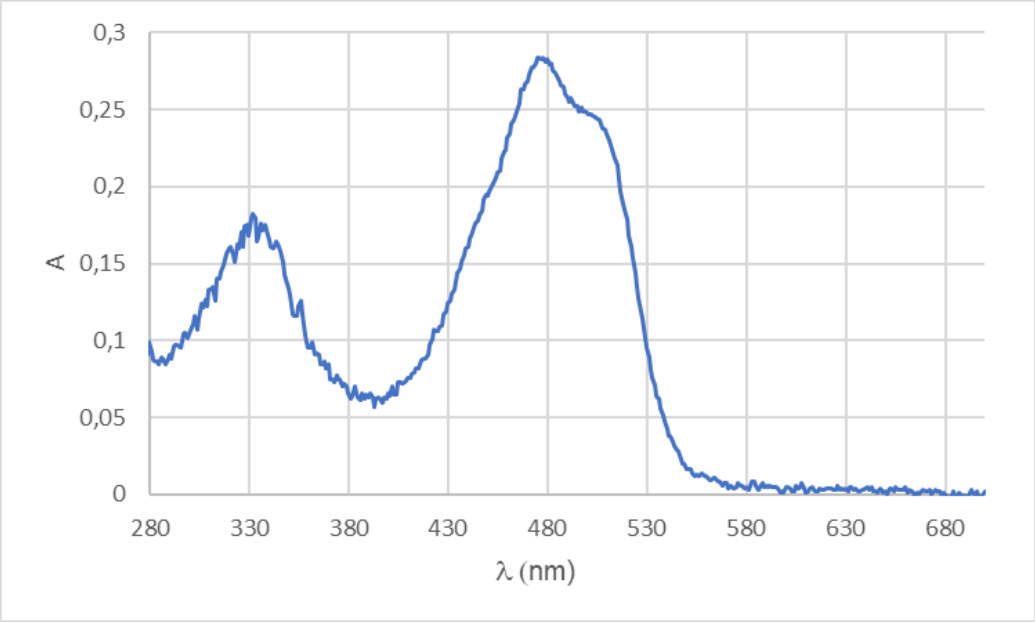
Complex **25**



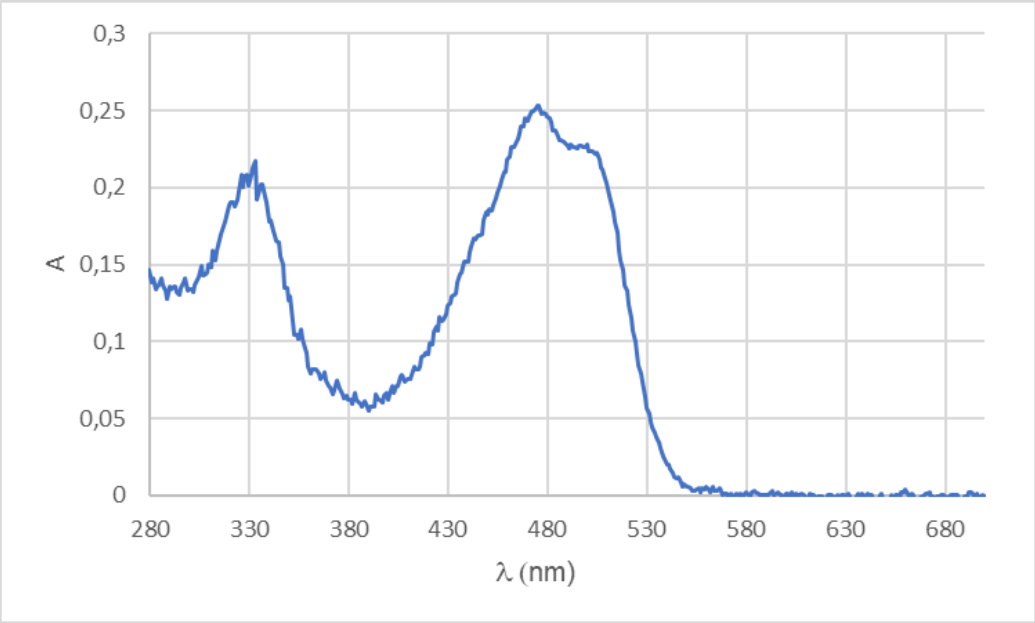
Complex **23**



Complex **7**

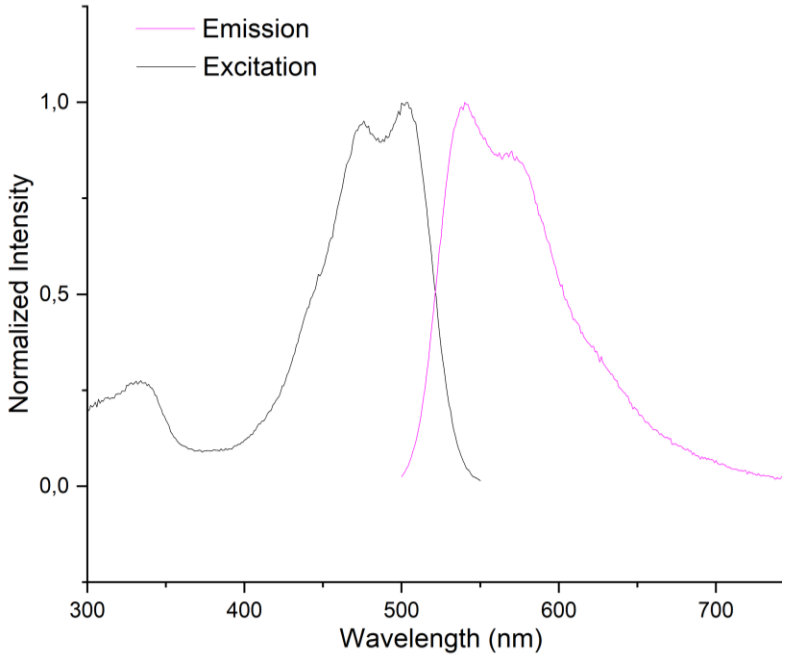


Complex **6**

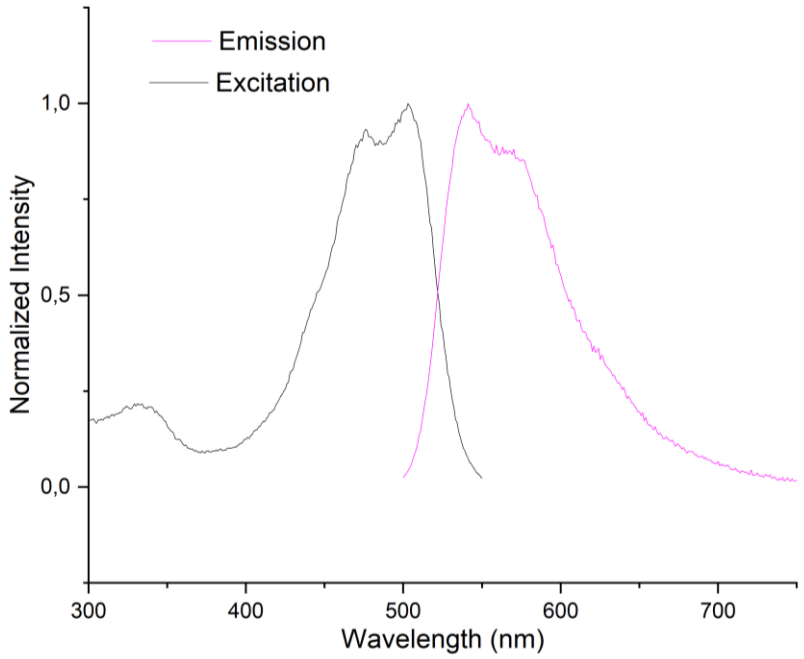


Excitation-Emission spectra

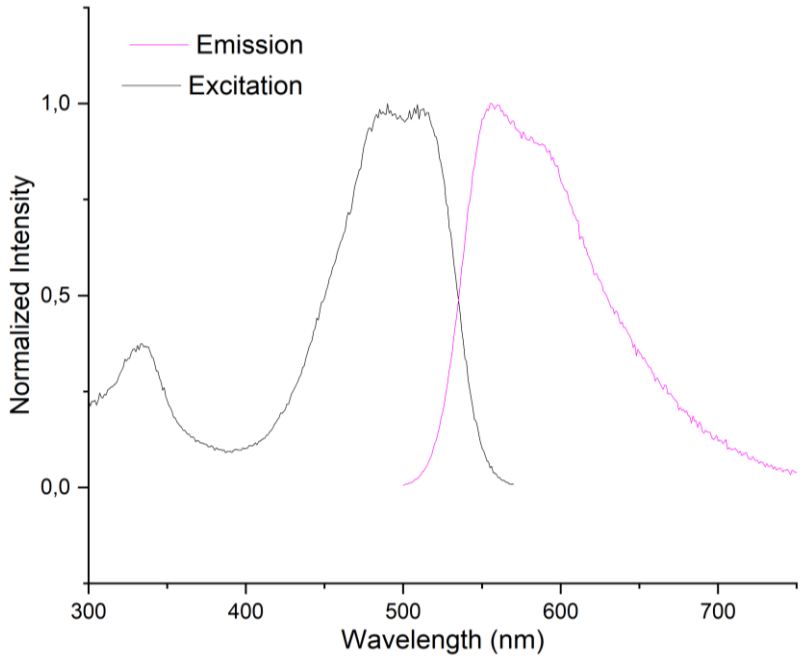
Complex **24**



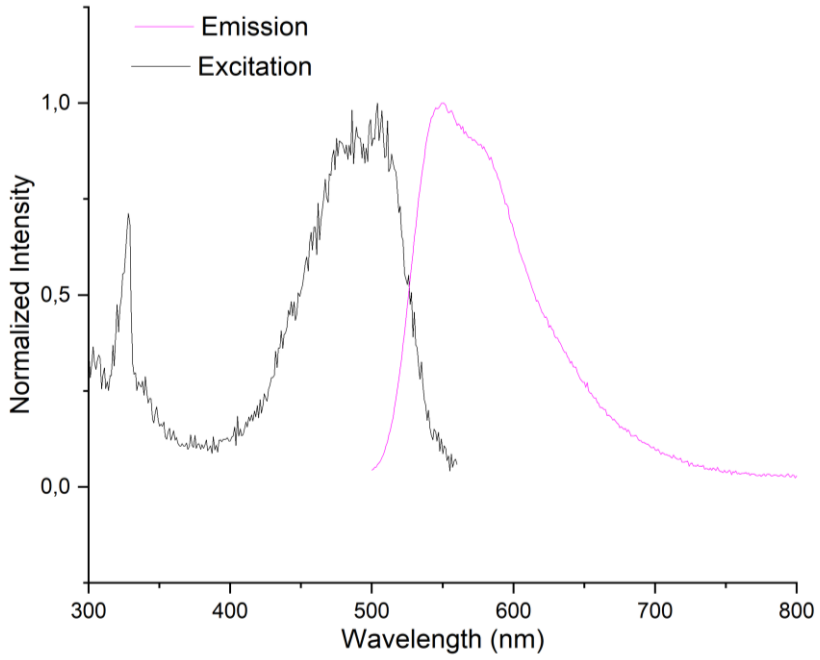
Complex **25**



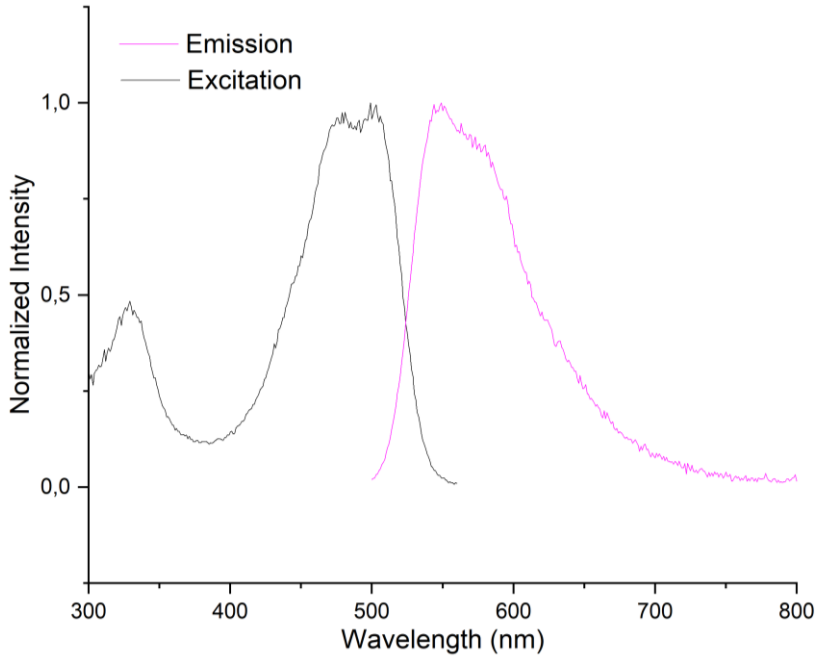
Complex **23**



Complex **7**



Complex **6**



References

1. https://robert.readthedocs.io

2. Intel(R). https://github.com/intel/scikit-learn-intelex

3. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., et al. (2011). Scikit-learn: Machine Learning in Python. J. Mach. Learn. Res. 12, 2825–2830.

4. Rücker, C., Rücker, G., and Meringer, M. (2007). y-Randomization and Its Variants in QSPR/QSAR. J. Chem. Inf. Model. *47*, 2345–2357. 10.1021/ci700157b.

5. Chuang, K.V., and Keiser, M.J. (2018). Comment on “Predicting reaction performance in C–N cross-coupling using machine learning.” Science *362*, eaat8603. 10.1126/science.aat8603.

6. Artrith, N., Butler, K.T., Coudert, F.-X., Han, S., Isayev, O., Jain, A., and Walsh, A. (2021). Best practices in machine learning for chemistry. Nat. Chem. 13, 505–508. 10.1038/s41557-021-00716-z.

7. Walsh, I., Fishman, D., Garcia-Gasulla, D., Titma, T., Pollastri, G., ELIXIR Machine Learning Focus Group, Harrow, J., Psomopoulos, F.E., Tosatto, S.C.E. (2021). DOME: recommendations for supervised machine learning validation in biology. Nat Methods 18, 1122–1127. 10.1038/s41592-021-01205-4.

8. Alegre-Requena, J.V., Sowndarya S. V., S., Pérez-Soto, R., Alturaifi, T.M., and Paton, R.S. (2023). AQME: Automated quantum mechanical environments for researchers and educators. WIREs Comput Mol Sci. 13, e1663.10.1002/wcms.1663.

9. Morrison, D.F. (1976). Multivariate statistical methods 2d ed. (McGraw-Hill).

10. Freund, Y., and Schapire, R.E. (1997). A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting. Journal of Computer and System Sciences 55, 119–139. 10.1006/jcss.1997.1504.

11. Hornik, K., Stinchcombe, M., and White, H. (1989). Multilayer feedforward networks are universal approximators. Neural Networks 2, 359–366. 10.1016/0893-6080(89)90020-8.

12. Dietterich, T.G. (2000). Ensemble Methods in Machine Learning. In Multiple Classifier Systems Lecture Notes in Computer Science. (Springer Berlin Heidelberg), pp. 1–15. 10.1007/3-540-45014-9_1.

13. European Organization For Nuclear Research and OpenAIRE (2013). Zenodo: Research. Shared. 10.25495/7GXK-RD71.

14. Bannwarth, C., Caldeweyher, E., Ehlert, S., Hansen, A., Pracht, P., Seibert, J., Spicher, S., and Grimme, S. (2021). Extended tight-binding quantum chemistry methods. WIREs Comput Mol Sci *11*, e1493. 10.1002/wcms.1493.

15. Delaney, J.S. (2004). ESOL: Estimating Aqueous Solubility Directly from Molecular Structure. J. Chem. Inf. Comput. Sci. 44, 1000–1005. 10.1021/ci034243x.

16. Friederich, P., Dos Passos Gomes, G., De Bin, R., Aspuru-Guzik, A., and Balcells, D. (2020). Machine learning dihydrogen activation in the chemical space surrounding Vaska's complex. Chem. Sci. *11*, 4584–4601. 10.1039/D0SC00445F.

17. Collado, S., Pueyo, A., Baudequin, C., Bischoff, L., Jiménez, A.I., Cativiela, C., Hoarau, C., and Urriolabeitia, E.P. (2018). Orthopalladation of GFP-Like Fluorophores Through C-H Bond Activation: Scope and Photophysical Properties. Eur. J. Org. Chem. 2018, 6158–6166. 10.1002/ejoc.201800966.

18. Laga, E., Dalmau, D., Arregui, S., Crespo, O., Jimenez, A.I., Pop, A., Silvestru, C., and Urriolabeitia, E.P. (2021). Fluorescent Orthopalladated Complexes of 4-Aryliden-5(4H)-oxazolones from the Kaede Protein: Synthesis and Characterization. Molecules 26, 1238. 10.3390/molecules26051238.

19. Garcia-Sanz, C., Andreu, A., de las Rivas, B., Jimenez, A.I., Pop, A., Silvestru, C., Urriolabeitia, E.P., and Palomo, J.M. (2021). Pd-Oxazolone complexes conjugated to an engineered enzyme: improving fluorescence and catalytic properties. Org. Biomol. Chem. 19, 2773–2783. 10.1039/D1OB00305D.

20. Dalmau, D., Crespo, O., Matxain, J.M., and Urriolabeitia, E.P. (2023). Fluorescence Amplification of Unsaturated Oxazolones Using Palladium: Photophysical and Computational Studies. Inorg. Chem. 62, 9792–9806. 10.1021/acs.inorgchem.3c00601.